

Survey paper

Deep reinforcement learning for network routing optimization: A systematic survey[☆]

Yuqian Song^a, Mao Ni^b, Jun Liu^{a,*}

^a School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China

^b China Mobile Group Device Co. Ltd., Beijing, China

ARTICLE INFO

Communicated by G. Bovenzi

Keywords:

Deep reinforcement learning
Routing optimization
Traffic engineering

ABSTRACT

Over the past decades, modern communication networks have undergone substantial expansion in both scale and complexity. Accordingly, traditional routing solutions increasingly struggle to meet the demands of these networks, as they often fail to adapt to the rising traffic dynamics and evolving quality-of-service (QoS) requirements. In response, deep reinforcement learning (DRL) has emerged as a revolutionary innovation in network routing optimization, benefiting the communication industry through more adaptive and intelligent routing strategies. In this paper, we first abstract a generic development life cycle for DRL-based routing algorithms. Then, we provide a comprehensive review of the applications of DRL in routing optimization, structured according to the stages within this life cycle. Finally, we present our perspectives on emerging research trends in DRL-based network routing optimization.

1. Introduction

Over the past decade, various 5 G applications, including enhanced mobile broadband (eMBB), ultra-reliable low-delay communications (URLLC), and massive machine-type communications (mMTC), have significantly transformed daily life [1]. With the advent of 6 G, communication services are anticipated to become more sophisticated and diversified [2]. As a result, computer networks, as the fundamental public infrastructure supporting human activities, are projected to encounter exponential increases in traffic complexity and escalating challenges in network control. To maintain acceptable service quality for end users, Internet service providers (ISPs) urgently require more advanced traffic engineering (TE) techniques to manage traffic flows effectively. TE encompasses a suite of techniques designed to optimize traffic distribution across switches, routers, and links in operational networks through policy optimization and performance evaluation. Theoretically, TE involves dynamically mapping real-time traffic status to appropriate network configurations, including three core tasks, i.e., network information collection, routing optimization, and routing configuration. Among these, the key point of TE lies in intelligently determining end-to-end traffic paths to efficiently accommodate user demands under

constrained resources. In this context, this survey specifically targets routing optimization in wired networks.

Traditional Interior Gateway Protocols (IGPs) include RIP [3], OSPF [4], and IS-IS [5] for shortest-path routing, as well as ECMP [6] for basic load balancing. While wired networks, such as ISP infrastructures, generally exhibit low topological dynamics, they are not entirely static. Link/node failures and maintenance operations frequently induce transient topology changes that temporarily increase network dynamics. In such scenarios, RIP employs periodic updates that lead to characteristically slow convergence. OSPF and IS-IS utilize link-state advertisement (LSA) flooding to propagate topology updates throughout the network. Although LSA propagation is relatively fast, it incurs substantial computational overhead and configuration complexity, potentially leading to link-state database (LSDB) explosion in large-scale networks. More critically, the dynamics in modern networks arise from time-varying traffic patterns, localized congestion bursts, and evolving multi-objective quality-of-service (QoS) requirements. In general, the above heuristic methods heavily depend on pre-configured rules, making them inadequate for adapting to these dynamic conditions [7]. Specifically, conventional IGPs typically only support equal-cost load balancing, where alternative paths with available but unequal

[☆] This work is partially supported by the National Natural Science Foundation of China under the Grant No. 62371057 and the “2023 AI Model Edge Deployment and Application Research Project” of China Mobile Group Device Co., Ltd.

* Corresponding author.

Email addresses: songyuqian@bupt.edu.cn (Y. Song), nimao@cmdc.chinamobile.com (M. Ni), liujun@bupt.edu.cn (J. Liu).

Table 1
Hierarchical classification for autonomous networks.

Level	L0 Manual Operations	L1 Assisted Operations	L2 Partial Autonomy	L3 Conditional Autonomy	L4 High Autonomy	L5 Full Autonomy
Execution	P	P/S	S	S	S	S
Perception	P	P/S	P/S	S	S	S
Analysis	P	P	P/S	P/S	S	S
Decision-making	P	P	P	P/S	S	S
Intent	P	P	P	P	P/S	S
Applicability	—	Selected scenes	Selected scenes	Selected scenes	Selected scenes	All scenes

^aP: Person-manual; S: System-autonomous

bandwidth remain unused, failing to efficiently utilize paths with heterogeneous bandwidth and resulting in poor link utilization (LU). Current solutions to handling QoS diversity rely on additional mechanisms, e.g., DiffServ, RSVP, and MPLS, yet these remain fundamentally static in nature. For more complex QoS optimization problems, some researchers have employed computationally intensive mathematical solvers that formulate routing as integer linear programming (ILP) or constraint programming (CP) problems solved using general-purpose optimizers like CPLEX [8] and Gurobi [9]. However, these mathematical solutions are limited to static network snapshots and require complete re-optimization when the network changes, leading to unacceptable response times that degrade service quality. Faced with exponentially growing traffic demands, ISPs cannot sustainably address these challenges through unlimited bandwidth expansion due to prohibitive infrastructure costs. As a result, network operators urgently need more intelligent routing strategies that can meet diverse QoS requirements within existing infrastructure constraints by adaptive traffic distribution.

As early as 2019, the TeleManagement Forum (TMF) formally introduced the concept of autonomous networks (AN), aiming to establish automated and intelligent operation and maintenance capabilities throughout the entire network lifecycle. TMF further defined a hierarchical classification for autonomous networks, categorizing their self-governing capabilities from the lowest to the highest levels (L0 to L5), as illustrated in the Table 1. Inspired by this novel concept, China Mobile, the largest telecommunications operator in China and a global leader in mobile communications and information services, has leveraged its extensive network infrastructure, vast user base, and rich data resources to pioneer innovations in AN. Central to this effort is the development of autonomous networks capable of “self-configuration, self-healing, and self-optimization,” with the ultimate objective of delivering a seamless “zero-wait, zero-fault, zero-touch” service experience. With the substantial advancements in artificial intelligence (AI) technologies, realizing this vision naturally calls for an AI-driven decision-making engine that can autonomously plan and execute operational tasks. Unlike conventional manual troubleshooting methods, this intelligent network system is expected to autonomously generate optimal configurations within milliseconds by processing real-time network data at scale, forming a fully closed-loop, end-to-end resolution framework.

Among various AI paradigms, deep learning (DL) has been employed in network routing tasks owing to its strong capability for real-time environmental sensing, as demonstrated in [10–13]. Given that network routing optimization typically involves well-defined objectives and evaluation metrics, DL in this context primarily refers to supervised learning rather than unsupervised learning. Nevertheless, DL-based methods often suffer from a heavy dependence on large-scale labeled datasets, limiting their scalability and adaptability. In parallel, reinforcement learning (RL)-based routing methods have been explored since the 1990s, as evidenced by [14–17]. As a bio-inspired ML method, RL mitigates the dataset dependency by learning through online interactions with the environment rather than relying on external supervision. However, RL-based methods encountered significant limitations when managing high-dimensional observation and decision spaces, particularly due to their reliance on tabular learning.

Recent advances have addressed these limitations by integrating powerful universal approximators, i.e., deep neural networks (DNNs), into RL, giving rise to deep reinforcement learning (DRL) [18]. By leveraging the strengths of both paradigms, DRL excels in continuous decision-making tasks through experience-driven learning. Following the groundbreaking success of DeepMind’s AlphaGo [19], the research community increasingly recognized the potential of DRL. In 2017, Stampa et al. [20] proposed the first DRL-based method for software-defined network (SDN) routing optimization, which has since stimulated the emergence of numerous DRL-based routing solutions. In general, using DRL for network routing optimization provides the following advantages:

- DRL enables network entities to learn and build knowledge of the networking environment without manual feature engineering or an explicit and complete mathematical model. In practice, accurately acquiring all relevant network parameters is a resource-intensive task that is intractable in large-scale or dynamic networks.
- Unlike conventional methods that rely heavily on expert knowledge, DRL agents are capable of discovering complex, sometimes counterintuitive, routing policies through a trial-and-error manner. This experience-driven learning enables the system to exceed the limitations of human intuition and manually crafted heuristics.
- DRL offers a flexible reward design mechanism that inherently supports the joint optimization of multiple performance objectives. In contrast, traditional IGP protocols are typically designed to optimize a single metric, such as hop count.
- DRL facilitates autonomous and adaptive routing decision-making for non-convex and complex network routing optimizations by leveraging temporal reasoning capabilities. It enables traffic to be dynamically distributed across appropriate links according to real-time network conditions, thereby maximizing resource utilization while proactively mitigating congestion.
- With the integration of deep neural networks, DRL can efficiently process high-dimensional inputs comprising hundreds of network parameters. This leads to significant improvements in computational efficiency and responsiveness, making DRL particularly suitable for time-sensitive applications such as millisecond-level packet forwarding.

In summary, DRL-based routing algorithms represent a transformative shift from “blind” traffic engineering to a proactive and intelligent control paradigm. By integrating real-time network sensing, dynamic routing decision-making engines, and feedback-driven control, DRL establishes a closed-loop adaptive routing system. This paradigm is particularly valuable in future scenarios with increasing demands from video streaming, industrial digitization, robotics, etc., where zero-packet-loss and low-delay transmission are crucial for achieving superior service agility and user experience (Table 2).

Several surveys have investigated the application of DRL to network routing optimization, such as the works in [21–25]. However, these surveys generally treated network routing as part of a broader research scope, where DRL-based routing was still in its early development stages, yielding limited contributions and relatively sparse coverage. In recent

Table 2
Comparison of related surveys.

Publication	Year	Scope	Taxonomy	DRL-based Routing Literature Coverage
[21]	2019	DRL for Communications and Networking	Core Network Functions	2 Papers
[22]	2021	ML for End-to-end Communication toward 6 G	Hierarchical Network Functions and ML Techniques	6 Papers
[23]	2021	ML for Routing Optimization in SDN	ML Techniques	19 Papers
[24]	2021	DRL for Traffic Engineering	Key Functional Areas	12 papers
[25]	2022	MADRL for future Internet	Core Network Functions	5 papers
Ours	2025	DRL for Network Routing	Phased Life Cycle	30 Papers

years, DRL-based routing optimization has evolved into a prominent research direction, characterized by extensive advancements across algorithmic design and application scenarios. Therefore, a dedicated and systematic survey is now essential to consolidate the state-of-the-art developments in this area. To this end, this paper proposes a structured development life cycle for DRL-based routing optimization, segmenting the research evolution into distinct stages. Based on this framework, we provide a comprehensive review of existing literature and detail the critical tasks associated with each stage. The main contributions of this survey are:

- We propose a development life cycle for DRL-based routing algorithms, providing a novel perspective to systematically investigate stage-specific research tasks.
- We conduct a comprehensive literature review on DRL applications for routing optimization, structuring the analysis according to the stages within the proposed life cycle.

- Based on the findings of this review, we offer insights into potential future research directions for DRL-based routing optimization.

The rest of this paper is organized as follows. [Section 2](#) provides an overview of DRL-based routing algorithms, including the development life cycle. [Sections 3–8](#) sequentially report considerations alongside solutions for each stage. [Section 9](#) presents future research directions and [Section 10](#) finally concludes this paper. [Fig. 1](#) illustrates the organization of this survey.

2. Deep reinforcement learning-based routing optimization: an overview

This section presents a foundational overview of DRL-based routing algorithms. We first introduce the preliminary concepts of DRL, followed by an explanation of its applicability to routing optimization. Finally, we describe the proposed development life cycle for DRL-based routing algorithms.

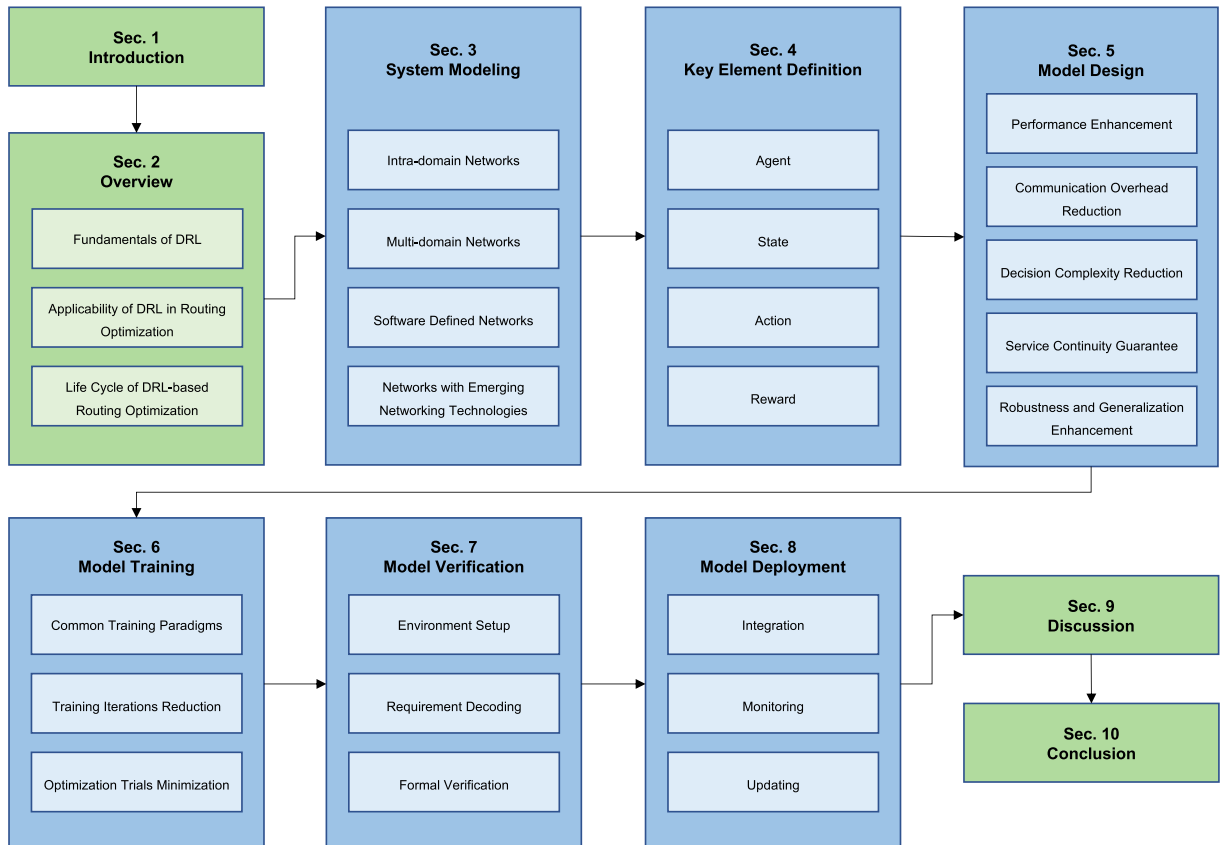


Fig. 1. Organization of this paper.

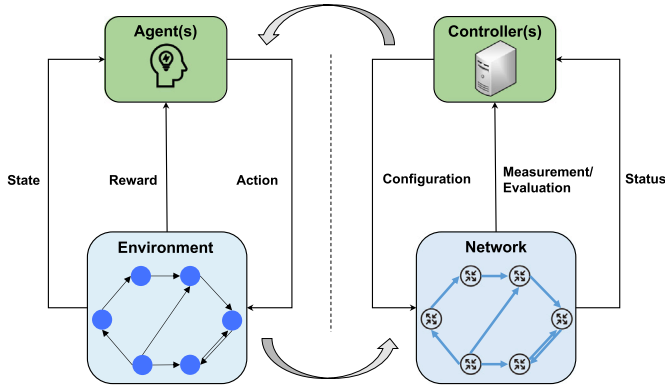


Fig. 2. Mapping of concepts between DRL and network routing.

2.1. Fundamentals of DRL

In DRL, a learning agent interacts with an environment to solve a sequential decision-making problem, accumulating intelligence through trial-and-error explorations guided by rewards. Since direct interaction with the real-life system incurs prohibitive expenditure, it is standard to employ a simulator that closely resembles the real world as the environment, serving as a more accessible platform for training. Such a learning process is modeled as a Markov Decision Process (MDP) [26] defined by a tuple $\langle S, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$, as shown in Fig. 2. Here, S and $\mathcal{A}$ denote the finite state and action space, respectively, $\mathcal{P} : S \times \mathcal{A} \rightarrow \mathcal{P}(S) \in [0, 1]$ represents the transition probability function mapping from state $s_t \in S$ to $s_{t+1} \in S$ upon executing a legal action $a_t \in \mathcal{A}$, and $\mathcal{R} : S \times \mathcal{A} \times S \rightarrow \mathbb{R}$ represents the reward function evaluating the selected action a_t and the state transition $s_t \rightarrow s_{t+1}$. During each decision interval, the agent (controlled by a policy $\pi : S \rightarrow \mathcal{A}$) will designate an action $a \in \mathcal{A}$ based on the given state $s \in S$, transitioning the system to a new state $s' \in S$ sampled from $\mathcal{P}$, and be rewarded with a $r(s, a)$ calculated via the reward function $\mathcal{R}$. Note that the transition probability function $\mathcal{P}$ that fundamentally describes the system dynamics is often unknown given the complexities involved in the mathematical modeling of real-world systems. As a result, most contemporary research studies choose to adopt model-free rather than model-based DRL methods [27] so that agents do not require prior system knowledge. For long-term optimization, the expected return of the agent involves discounted cumulative rewards, expressed as

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t(s_t, a_t) \right], \quad (1)$$

where $\gamma \in (0, 1)$ is a discount factor that balances immediate rewards versus future rewards. T represents the time scale, which can be either infinite or finite, corresponding to infinite-horizon or episodic tasks, respectively. Under a given policy π , both state-value function $V^{\pi}(s)$ or Q-value function $Q^{\pi}(s, a)$ can evaluate the expected return starting from any state s_t or state-action pair (s_t, a_t) as follows:

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t(s_t, a_t) | s_0 = s \right], \quad (2)$$

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t(s_t, a_t) | s_0 = s, a_0 = a \right]. \quad (3)$$

Based on the agent's learned objective, model-free DRL methods can be categorized into value-based and policy-based.

In value-based methods, the agent aims to learn an approximation of the value function. The optimal policy π^* is derived by acting greedily with respect to the function. In this case, Q-value $Q^{\pi}(s, a)$ iterations primarily abide by the Temporal Difference (TD) principle that merges

insights from Monte Carlo (MC) and Dynamic Programming (DP) methods. Specifically, MC methods suggest approximating the Q-value by averaging the rewards received after visiting the state s until the conclusion of an episode. On the other hand, DP methods update the estimated values of states by solving the Bellman optimality equation, which incorporates the estimated values of the next states. This practice is known as bootstrapping. To overcome the episodic constraint of MC methods and the demanding requirement of DP methods regarding the transition function, the mathematical recursive form of TD methods is derived as

$$Q^{\pi}(s, a) \leftarrow (1 - \alpha)Q^{\pi}(s, a) + \alpha \left[r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q^{\pi}(s', a') \right], \quad (4)$$

where α is the learning rate, influencing the pace of the learning process.

Alternatively, policy-based methods learn a parameterized policy without resorting to the value function. This practice remedies a severe deficiency of value-based methods that can only deal with discrete action space. In general, a stochastic policy maps an arbitrary state to an action distribution, i.e., $P(a|s) = \pi_{\theta}(a|s)$, and a deterministic policy maps the observed state to an exact action, i.e., $a = \pi_{\theta}(s)$. According to the Policy Gradient (PG) theorem [28], the parameters θ can be optimized by performing gradient ascent on the expected return. The policy gradients are expressed as

$$\nabla J(\theta) = \mathbb{E}_{s \sim \rho^{\pi_{\theta}}, a \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) \right], \quad (5)$$

where $\rho^{\pi_{\theta}}$ denotes the state distribution under policy π_{θ} , i.e., the samples used for updates are collected under the current policy, identified as an on-policy method. In this case, all samples collected at the current iteration are not reusable for future updates, thus rendering a prominent deficiency of low sample efficiency. In contrast, off-policy methods, where the policy collecting experience differs from that being optimized, excel at sample efficiency and potentially promote exploration.

In addition, policy-based methods have a well-known variant named the Actor-Critic (AC) method, featuring a PG-based actor network for behavior inference and a critic network for value approximation. During the learning process, the actor network is refined based on the value estimated by the critic network, and the critic network is updated based on the policy generated from the actor network. By merging the direct policy optimization of policy-based methods with the stability and efficiency of value estimation from value-based methods, AC methods have become foundational frameworks in the evolution of sophisticated DRL methods.

2.2. Applicability of DRL in routing optimization

As illustrated in Fig. 2, the nature of DRL lies in the interaction between the agent and the environment, where the agent observes states, executes actions, and receives rewards. The environment typically represents the external world in which the agent operates. During interactions, the agent makes sequential decisions and optimizes its policy by maximizing the cumulative reward. Similarly, routing optimization refers to an iterative process during which the controller continuously monitors the network, deploys routing configurations, evaluates the operational effectiveness through network measurements, and adjusts the routing strategy until achieving desirable network performance. In conclusion, the patterns observed in DRL and the routing optimization process are highly symmetrical, demonstrating the applicability of DRL in addressing routing optimization problems. Based on their analogy, we formally introduce a general interaction process of DRL-based routing algorithms by mapping concepts between DRL and routing.

Assume a network environment where an existing network device or an extra deployed controller functions as a DRL agent. At the beginning of each interaction, the agent acquires a representation of the current network status through proactive perception or passive acquisition from measurement devices. Owing to real-world factors encompassing physical, temporal, and financial limitations, the acquired status is prone

to being partial. The embedded policy network of the agent then maps the status to a corresponding action, such as certain network operations or routing configurations. After the execution of the action, the network environment would be influenced and transition to a new status with a certain probability. Subsequently, the agent receives a numerical delayed reward that evaluates the latest action under the associated network status by measuring specified network performance metrics. Note that the objective of the DRL agent is to maximize the long-term discounted cumulative reward. Therefore, the representation of the reward plays a critical role in shaping the agent's behavioral preferences and determining the final network performance.

2.3. Life cycle of DRL-based routing optimization

The successful application of DRL-based routing algorithms hinges on a well-structured life cycle that incorporates critical functional abstractions, enabling researchers to anticipate and address potential tasks proactively. While a standardized development process for DRL-based routing models remains absent, extensive research has explored methodologies for ML implementation, such as Knowledge Discovery in Databases (KDD) [29], Cross-Industry Standard Process Model (CRISP-DM) [30,31], and Team Data Science Process (TDSP) [32]. Motivated by these findings, we propose a life cycle describing the comprehensive process of applying DRL-based algorithms to network routing optimization. This life cycle comprises five key stages: system modeling, model design, model training, model verification, and model deployment. Some stages may be further subdivided into smaller steps. Note that the sequence presented here is indicative rather than prescriptive. These stages can occur sequentially, concurrently, or even in combination. We will now provide an overview of each stage as follows. Table 3 provides a stage-wise summary of the issues and concerns we discuss in this survey.

2.3.1. System modeling

Although DRL bypasses the need for precise network modeling and extensive labeled datasets, each DRL-based routing solution must establish a mathematical representation of the network as the environment before initiating algorithm design. Typically, the network is modeled as a graph $G = \{\mathcal{V}, \mathcal{E}\}$. $\mathcal{V}$ denotes the set of forwarding entities, and $\mathcal{E}$

denotes the set of links. To better reflect the complexity of real-world networks, current research primarily focuses on the following scenarios: intra-domain networks, multi-domain networks, SDN, and networks integrated with more advanced networking technologies.

2.3.2. Key element definition

At this stage, researchers must define the components on the learning side to construct the DRL-based decision model (as illustrated in Fig. 2), including the agent, state, action, and reward. Typically, the state space may comprise the traffic matrix [33], node sets, link characteristics, or session-level attributes; actions are generally defined as paths, link weights, next-hop, or split ratios; the reward function reflects the target objectives, often aligned with key performance indicators such as end-to-end delay, packet loss rate, throughput, and maximum link utilization (MLU) across the network.

2.3.3. Model design

Beyond the foundational algorithmic models, researchers are expected to introduce further enhancements to unlock the full potential of DRL-based routing while addressing practical constraints. Our literature review reveals a prevailing consensus in the research community regarding performance advantages, communication overhead reduction, decision complexity reduction, uninterrupted service delivery, as well as robustness and generalization enhancement (Fig. 3).

2.3.4. Model training

The model training stage encompasses two essential functions: feeding agents with valuable experience to learn the optimal behavior patterns and systematically optimizing hyper-parameter settings. Due to the inherently low exploration efficiency of model-free DRL paradigms and the complexity of network routing, this stage may require substantial time and computational resources, leading to significant economic costs that overwhelm many research institutions and companies.

2.3.5. Model verification

Model verification is essential in any software development cycle, as it ensures the quality of the produced model and minimizes future maintenance costs. Due to commercial and operational constraints, most verification processes proceed within simulation environments rather than real-world networks. In this context, researchers should strive to replicate real-world conditions during simulation using high-fidelity simulators. As a potential production software, DRL-based routing solutions must demonstrate practicality-oriented attributes in addition to clear performance advantages. Five main metrics are identified in the reviewed papers: reproducibility, cost efficiency, robustness, generalization capabilities, and partial deployability.

2.3.6. Model deployment

Verified routing solutions hold potential for practical deployment. However, due to commercial constraints, most existing studies provide only proof of concept, lacking the specific details required for practical implementation. In theory, this stage encompasses three main activities: software integration, monitoring, and updating, all aimed at achieving long-term performance advantages.

3. System modeling

In this section, we present four commonly adopted network modeling methodologies in DRL-based routing algorithms, regarding intra-domain networks, multi-domain networks, SDN, and networks integrated with emerging networking technologies. The system models of reviewed DRL-based routing algorithms are listed in Tables 4 and 5.

3.1. Intra-domain networks

Technological advancements in networking have significantly expanded network coverage over the past decades. As a result, successive generations of institutional, personal, and mobile computers have been

Table 3

All considerations, issues, and challenges explored in this survey.

Stage	Considerations, Issues, and Challenges
System Modeling	Modeling Methodologies by Domain Scope and SDN Attack-Aware SDN Routing Modeling Service-Aware Routing Modeling
Key Element Design	Centralized vs. Distributed Agent Deployment Strategies Univocal vs. Multi-Dimensional State Space Design Consistent vs. Scalable Routing Actions Global vs. Local vs. Neighbor Reward Functions
Model Design	Underutilized Agent Decision Capabilities Heavy Information Exchange in Resource-constrained Environments Cost and Delay Overheads from High-Dimensional Decision Spaces Risk of Service Disruption from Suboptimal Routing Strategies Potentially costly retraining under network changes
Model Training	Common Cooperative Training Paradigms Excessive Early-stage Exploration without Prior Knowledge Iterative Hyperparameter Tuning under Unknown Search Space
Model Verification	Faithful Simulation Environments Despite Non-negligible Modeling Gaps Formal Verification of General and Business-driven Metrics
Model Deployment	Full vs. Incremental Deployment Feedback Monitoring Loops for Detection Secure and Efficient Retraining

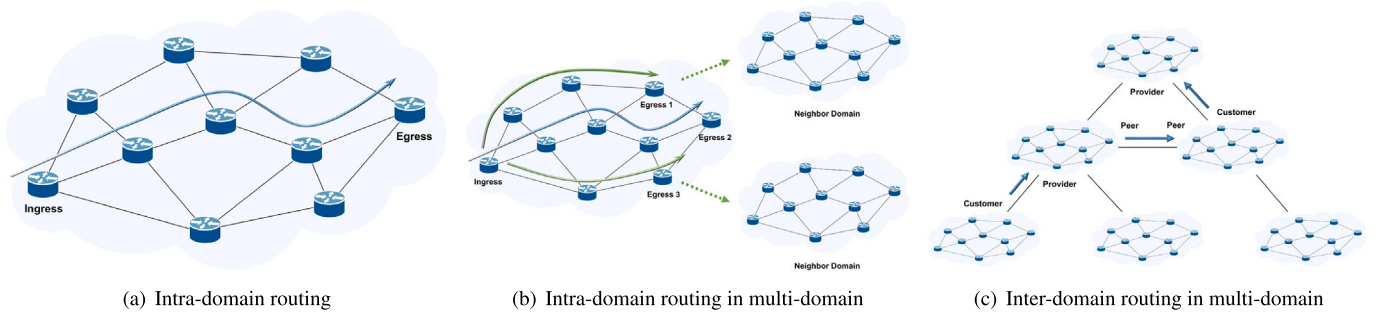


Fig. 3. Illustrations of three stages for domain-related routing.

Table 4

System model and key elements of reviewed research works.

Ref.	Approach	Objective	System Model	Agent	State	Action	Reward
[20]	DPG	Low delay	SDN	SDN controller	Traffic matrix	Link weights	- Mean network delay
[37]	DDPG	Low MLU	Multi-domain network	Domain controllers	LU	Split ratios over paths	Local & neighbors' MLU
[38]	AC	High throughput	Multi-domain network	Domain controllers	S-d pairs, LU, & last throughput	Next hop	Average throughput
[41]	DDPG	Low delay	SDN	SDN controller	Traffic matrix	Link weights	- Mean network delay
[42]	DDPG	Low delay	SDN	SDN controller	Traffic matrix	Link weights	- Mean network delay
[43]	DDPG	Low delay & loss	SDN	SDN controller	Traffic matrix	Link weights	Delay & packet loss
[45]	DDPG	Low delay	SDN	SDN controller	Traffic matrix	Link weights	- Mean network delay
[46]	DQN	Low delay & loss, high bandwidth	SDN	SDN controller	S-d pairs	Path	Bandwidth, delay, & packet loss
[47]	DDPG	High QoS & security	SDN	SDN controller	Traffic demands, flow table & channel occupancy rates	Next hop	Attack & QoS reward
[50]	DDPG	Low delay & loss, high throughput	Multi-domain with SDN	Domain & root controllers	Link status	Link weights	Delay & packet loss
[53]	PPO	High utility	SDN	Routers	Flow features, topology information, resource usage, & node indicator	Next hop	Service utility
[54]	AC	High utility & satisfaction	SDN with NS	SDN controller	Traffic demands, link bandwidth, link packet loss, & link delay	Paths & bandwidth	Network utility & service satisfaction
[55]	PG	High schedulability	TSN	Controller	Link status	Scheduled flow & its route	Schedulability
[57]	PPO	Timely delivery	WAN	Routers	Flow, routing, & network information	Next hop & deadline allocation	Boolean
[59]	DQN	Low delay	Intra-domain network	Routers	Current & neighbor nodes, first packet's destination	Next hop	- Local delay

interconnected, leading to diverse application services and a substantial proliferation of traffic data. To manage the explosive growth in data volume across increasingly large-scale networks, the concept of a “domain” has been introduced, denoting a logical boundary that defines a set of devices and resources administered by a common set of control entities. In this context, an intra-domain network constitutes the minimal autonomous unit capable of independent routing optimization. Typically modeled as a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, the vertex set $\mathcal{V} = \{1, \dots, V\}$ represents the network nodes, while the edge set $\mathcal{E} = \{1, \dots, E\}$ represents the communication links. Each node is equipped with a dedicated buffer to temporarily store packets awaiting transmission, where packets are processed according to specific queuing disciplines, such as first-in-first-out (FIFO) [34], priority queuing [35], fair queuing [36], and weighted fair queuing [34]. Similarly, each link is associated with a designated bandwidth and propagation delay. Since routing fundamentally involves end-to-end decision-making to forward data packets from source to destination nodes via intermediate nodes, its performance is closely tied to queuing delays within nodes and transmission delays across links.

3.2. Multi-domain networks

Although physical network resources are isolated across different domains, inter-domain traffic transmission remains inevitable, introducing

additional complexities. In such multi-domain networks, three types of routing problems arise: (i) intra-domain routing for traffic terminated in the current domain, (ii) intra-domain routing for traffic destined to other domains, and (iii) inter-domain routing. The work in [37] concentrates on the first two types of problems, abstracting multiple domains as sub-graphs interconnected by peering edges, with designated “border nodes” serving as endpoints for inter-domain communications. Within each domain, the ingress node serves as both the source for local demands and the border for external demands. Similarly, the egress node acts as the destination for internal traffic and the exit point for outbound traffic. In alignment with practical implementation where distinct algorithms are applied to interior and border routers, terminal and outgoing demands are managed separately. For convenience, inter-domain routing in this case is configured using classical shortest-path mechanisms. On the other hand, the work in [38] proposes the first DRL-based model for inter-domain routing, built upon the operational principles of the Border Gateway Protocol (BGP) [39]. In their model, domains form customer-provider or peer-to-peer relationships. Customers announce available routes to providers, who then aggregate and propagate this information, along with peer-learned paths, to their own customers. Besides, next-hop selection is governed by a priority hierarchy, preferring routes learned from customers over those from peers and providers. These two works represent pioneering attempts at DRL-based routing algorithms

Table 5
System model and key elements of reviewed research works.

Ref.	Approach	Objective	System Model	Agent	State	Action	Reward
[60]	DDPG	High utility	Intra-domain network	Controller	Sessions' throughput & delay	Split ratios over paths	Sessions' utility
[61]	AC	Low MLU	Intra-domain network	Routers	Traffic demands & LU	Split ratios	- MLU
[62]	DDPG	Low MLU	Intra-domain network	Routers	Traffic demands, historical LU & action	Split ratios	- MLU
[63]	AC	Low MLU	Intra-domain network	Routers	Historical traffic demands, LU & action	Split ratios	- MLU
[64]	AC	Low MLU	Intra-domain network	Routers	Traffic demands & LU	Split ratios	- MLU of distinct ranges
[65]	DQN	Low delay	Intra-domain network	Routers	Current & future packets' destinations, historical action, max-queue neighbor	Next hop	- Local delay
[66]	DDPG	Low MLU	SDN	Switches	LU	Split ratios	MLU-related
[67]	PPO	Low delay	Intra-domain network	Controller	Traffic matrix	Link weights	- Mean network delay
[68]	PPO	Low MLU	SDN	SDN controller	LU	Split ratios	MLU difference across steps
[69]	PPO	Low MLU	Intra-domain network	Controller	Traffic demands	Link weights	MLU-related
[70]	DDPG	Fast flow completion	SDN	SDN controller	Traffic matrix	Link weights	Average flow completion time
[71]	ACKTR	Low delay & high throughput	Intra-domain network	Controller	Link bandwidth, link delay	Path	Throughput & delay
[72]	PPO	Low delay & loss, high throughput	SDN	Routers	S-d pairs, & topology	Next hop	Throughput, delay, & packet loss
[73]	DQN	Attack prevention	Intra-domain network	Controller	Traffic & topology	Path	Attack reward & cost
[74]	PPO	Low delay & loss	Intra-domain network	Controller	Link status	Link weights	Delay & packet loss

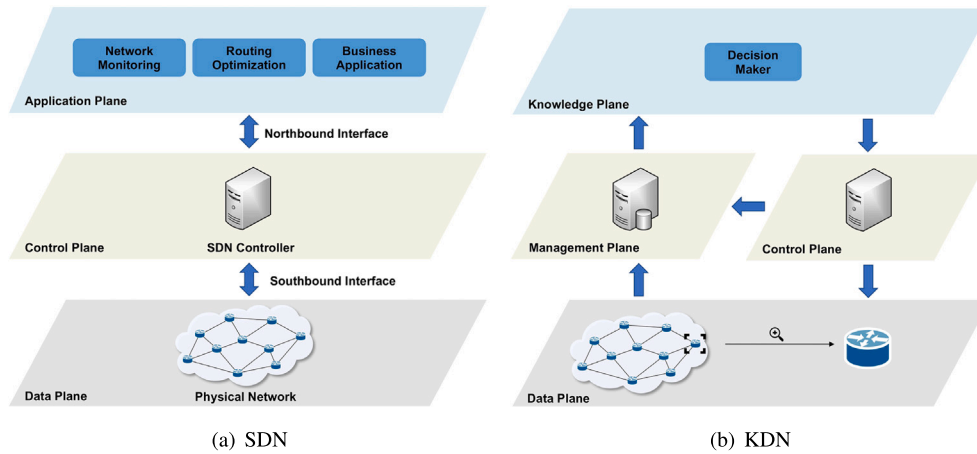


Fig. 4. Illustrations of SDN and KDN architectures.

in multi-domain networks, which have drawn enormous attention from academia and are anticipated to inspire future researchers to devise more advanced DRL-based routing algorithms in this context.

3.3. Software defined networks

Traditional implementation of routing algorithms relies on time-consuming and error-prone manual configuration, contributing to rigid routing policies that do not adapt well to dynamic real-world networks. Recently, SDN has emerged as a viable network paradigm to address this issue, as illustrated in Fig. 4(a). By decoupling the data and control planes, SDN effectively alleviates the configuration complexity through centralized and programmable control, which is conducive to agile routing configuration. Moreover, the software-driven control plane allows for logically centralized network intelligence, providing an ideal platform for integrating DRL techniques and prompting a new networking paradigm known as knowledge-defined networking (KDN) [40]. Built on SDN, KDN further incorporates the management and knowledge planes, forming a closed-loop control system as depicted in Fig. 4(b). While SDN is a more universal term in the networking field compared to KDN, we

primarily use SDN throughout this survey to avoid redundancy unless specified otherwise.

The works in [20,41,42] focused on routing optimization for the standard three-layer SDN architecture, including (i) data plane, comprising packet forwarding devices; (ii) control plane, responsible for global network monitoring and DRL-based path computation; and (iii) application plane, serving as an interface for high-level network applications. Communication between planes is typically facilitated by the OpenFlow protocol.

Exemplifying an initial realization of the KDN paradigm, the work in [43] constructs the data plane and control plane based on a conventional SDN architecture, and further introduces an AI plane to accommodate the DRL agent. Specifically, the data plane consists of OpenFlow-enabled devices, which offer fine-grained network status collection and support online updates of forwarding rules. The control plane, acting as an intermediary platform, connects to the data plane via a southbound interface and to the AI plane via a northbound interface. Through the southbound interface, the controller collects comprehensive network status information, including flow table statistics, resource availability, and QoS measurements, which are essential for evaluating network performance.

Via the northbound interface, the controller then conveys a global network view to the AI plane, where the DRL agent processes this data and outputs a new set of link weights. It is noteworthy that the granularity and timeliness of network status collection significantly influence the performance of the DRL agent. According to the sampling theorem, status reporting frequency must be at least twice the dominant frequency of traffic variations to capture network dynamics accurately. Various monitoring methods, including pushing, polling, and in-band network telemetry (INT), are used in SDNs. Pushing methods incur low computational overhead but offer coarse-grained information. Polling methods provide flexible and on-demand network monitoring, albeit with limitations in the format of retrievable information. INT techniques embed network state information (e.g., link utilization and delay) directly into packets at each capable switch, enabling fine-grained, real-time monitoring but imposing significant processing overhead on the controller. In [43], a polling-based monitoring method is adopted. Subsequently, the controller translates the inferred link weights into flow tables and installs them on switches via the southbound interface. To ensure consistency during flow table updates, the authors use an intermediate transfer method [44].

In contrast, the works in [45,46] demonstrate a complete realization of the KDN paradigm. As described in [45], in KDN, the data plane consists of line-rate programmable forwarding hardware responsible for storing, forwarding, and processing data packets, without awareness of the global network state and relying on the control and management planes for forwarding table population and configuration updates. The control plane exchanges operational states to instruct the data plane on forwarding and processing policies. Meanwhile, the management plane ensures the long-term operation and performance of the network by monitoring and providing network analytics. Leveraging information from both the control and management planes, the knowledge plane, i.e., the decision-making agent, is allowed to have a comprehensive understanding and control of the network and infers the optimal routing strategy. At a more granular level, the work in [46] devises a routing architecture composed of a topology module, a monitoring module, a processing module, a routing module (i.e., agent), and an installation module. During operation, the topology module responds to queries with statistical information collected at switch ports, while the monitoring module, establishes the topology as a set of tuples and gathers statistical information by periodically exchanging messages with the topology module. The processing module then retrieves raw collected data to compute metrics that serve as input features for the routing module, which learns network behavior and generates routing plans. Finally, the installation module proactively installs or updates flow entries in the switches based on the computed routing paths, ensuring flow tables are refreshed in time. To summarize, this architecture aligns with the KDN concept, where the topology module represents the data plane, the monitoring and installation modules operate within the control plane, the processing module corresponds to the management plane, and the routing module embodies the knowledge plane, leveraging both collected network information and DRL capabilities for intelligent decision-making.

In addition, while SDN simplifies network configuration by decoupling the data and control planes, this shift leaves physical forwarding devices unintelligent, making them vulnerable to exploitation by attackers. In this regard, the work in [47] models a SDN network susceptible to both external and internal cyberattacks, including distributed denial-of-service (DDoS) [48] and gray hole attacks [49]. In a DDoS attack, fraudulent flow requests overwhelm the network, while in a gray hole attack, compromised forwarding devices intentionally drop legitimate traffic. Without timely intervention, these attacks can lead to significant packet loss, increased delay, or switch failures, highlighting the need for fine-grained intrusion detection and routing optimization using DRL. Furthermore, the work in [50] investigates DDoS attacks in multi-domain networks managed under the SDN architecture. To mitigate the risk of DDoS-induced failures across both intra-domain and inter-domain, this work proposes a hierarchical and centralized

SDN-based management framework. In this design, each network domain is governed by a local controller responsible for intra-domain information gathering and routing decisions. Then, a central root controller coordinates inter-domain path selection and enables parallel computation to accelerate path planning across domains. Simulation results demonstrate that this approach effectively enhances the adaptability and stability of the network in the face of cyberattacks.

3.4. Networks with emerging networking technologies

The Internet typically hosts several end-to-end communication sessions that transmit simultaneously. For simplicity, these sessions are often modeled as undifferentiated service flows. However, differentiated services and QoS routing have become increasingly important for two reasons. First, emerging applications, such as live streaming, immersive virtual reality (IVR), online gaming, and industrial control, have raised the demand for distinct and stringent QoS requirements (e.g., delay, jitter, throughput, and packet loss rate). Second, advances in flow classification technologies [51], SDN, and programmable network devices [52] enable more precise flow identification and routing control. In this regard, a common method encodes flow types into state inputs [53]. In contrast, the work in [54] uses network slicing (NS) to realize logically independent operation, reducing competition and allowing separate routing decision-making. Moreover, delay-sensitive applications, particularly in industrial Internet of Things (IIoT) scenarios, have become more prominent. Traditional Internet, however, can only provide best-effort (BE) services due to unpredictable queuing delays. In response, the work in [55] explores time-sensitive networking (TSN) [56], which is typically modeled as a hybrid traffic system consisting of three classes: time-triggered (TT), audio-video bridging (AVB), and best-effort (BE). TT traffic requires deterministic delay, AVB allows bounded delays, and BE traffic has no timing guarantees. In theory, TSN ensures low delay and zero jitter through a globally synchronized clock and gate control list (GCL) that reserves time slots for critical traffic. Upon receiving a flow request, the controller directs it to the appropriate priority queue based on QoS requirements. Within the node, higher-priority traffic is transmitted first, and flows with equal priority follow the first-come-first-served (FCFS) principle. In this case, the agent not only determines forwarding decisions but also schedules flows before routing. Compared to random strategies, linear programming, general-purpose metaheuristic search methods, and domain-specific expert heuristics in TSN scheduling, the integration of DRL significantly enhances flow schedulability as demonstrated by simulation results.

It is worth noting that TSN primarily operates at Layer 2 of the network stack, where TT traffic typically exhibits strong periodic characteristics. In contrast, Wide Area Networks (WANs) feature statistical multiplexing and inherently non-periodic traffic patterns, making the direct application of existing deterministic networking approaches highly challenging. As a solution, the work in [57] first proposes a novel coordinated earliest deadline first scheduling scheme. Then, it derives the single-hop and end-to-end deadline satisfiability criteria to precisely estimate the capacity of deadline-constrained traffic that deterministic networking (Detnet) can satisfy. Finally, it formulates the problem of deadline-aware routing and scheduling as a stochastic optimization task and employs DRL to enable real-time decision-making under dynamic network conditions. Compared to TSN, achieving deterministic transmission in WAN environments poses a significantly greater challenge, meriting deeper investigation and specialized solutions.

Summary: This section first presents mainstream modeling methodologies for two foundational and widely adopted network architectures: domain-related networks (including single-domain, multi-domain, and cross-domain scenarios) and SDN. In contrast to domain-related networks, which typically rely on graph-based abstraction, SDN is often characterized by the integration of multiple functional modules forming a closed-loop decision-making process. Moreover, we underscored the security vulnerabilities of SDN that arise from the decoupling of

control and data planes, exposing the system to a broad spectrum of cyberattacks and necessitating comprehensive attack modeling to ensure secure and robust routing. In addition, we discussed network slicing as a complementary technique that enables logically independent optimization across coexisting traffic flows. Finally, we introduced TSN and Detnet, critical enablers in the IIoT, where DRL-based algorithms are well-positioned to realize strict delay guarantees even under the coexistence of time-critical and best-effort traffic.

4. Key element definition

With the environment modeling completed, this section systematically defines the remaining components of the DRL interaction, including agent, state, action, and reward, to finalize the interaction loop of the DRL-based routing problem. The key elements of reviewed DRL-based routing algorithms are listed in Table 4 and Table 5.

4.1. Agent

Due to deployment constraints, most studies implement agent(s) on existing network infrastructures. Generally, contemporary network architectures are categorized as centralized or distributed, corresponding to single-agent and multi-agent DRL paradigms, respectively.

Centralized architectures traditionally employ a single controller (i.e., agent) for unified global control, enabling consistent management but posing significant configuration challenges. With the emergence of SDN, the DRL-driven SDN controller has become the preferred agent for scalable centralized control. However, the dimensions of state and action may grow exponentially with network expansion, demanding high-performance memory and computational units. Given resource-limited hardware devices in practice, the work in [53] proposes a MADRL-based routing approach for SDN. In this approach, each forwarding node is equipped with an agent with individual observation and action space, and all agents centrally reside on the SDN controller, effectively relieving the processing workload.

In recent years, the network architecture has gradually transformed from centralized to distributed for improved scalability and resource utilization [58]. In this context, functionalities, computational capabilities, and memory resources are distributed across multiple network devices, facilitating the implementation of multi-agent algorithms. By empowering forwarding nodes with real-time decision-making abilities, MADRL-based algorithms provide an enhanced response while constraining the scope of negative impact to a single node in the face of training failures or local optima. Multi-domain networks represent another typical application of MADRL, where each domain functions as an agent, as evidenced by the work in [38]. In particular, two agents are established for each domain in [37] for differentiated traffic administration.

4.2. State

The state represents the accessible network status to the agent, serving as the foundation of decision-making. Based on our research findings, existing DRL-based routing algorithms commonly utilize the following categories of information to depict the network:

- Traffic matrix: Traffic Matrix (TM) can intuitively reflect the traffic volume between each potential source-destination (s-d) pair in the network, standing out as the most fundamental state definition, as demonstrated by works in [20,41,42,45].
- Set of nodes: In this context, agent(s) only observe the indices of some specific nodes to make decisions, such as the source, destination, and neighboring nodes. Such node-set state formulation is concise yet considers a well-established topology as a prerequisite for index identification. For example, the work in [46] defines the state space as the set of source-destination node pairs, and the state of the agent in [59] includes the current node, the neighboring nodes of the current node, and the destination node of the first

packet in the current node's queue, which is represented by three one-hot vectors.

- Set of link characteristics: Link bandwidth utilization is a quantitative measure that reflects the link usage status, which is particularly crucial in tasks related to congestion and load balancing. In essence, link utilization is determined jointly by traffic demands on the network side and routing decisions made by agents. These dual influences facilitate link utilization to provide valuable insights for decision-making processes, adopted by works in [37,38,54].
- Set of session characteristics: Inherited from Network Utility Maximization (NUM), some researchers concentrate on session-specific utility instead of global network performance metrics. Accordingly, the observation is formulated as session-specific performance indicators or other pertinent information, as demonstrated by the work in [60].

While single-faceted network observations are concise and compact, they may be insufficient in more complex scenarios. Consequently, researchers often seek to provide agents with broader observations to enhance environmental awareness. For instance, the works in [61–64] integrate flow demands, link utilizations, and forwarding actions (i.e., next-hop indices) across multiple temporal scales into the observation space. In [53], flow characteristics, topology information, and resource utilization metrics are combined to construct a comprehensive network view. Moreover, a conditional state vector is introduced to emphasize nodes along the established routing path, which is asserted to improve routing decision quality.

4.3. Action

The action space defines the set of feasible actions that govern routing configuration patterns. Various formulations have been adopted in DRL-based routing algorithms, including:

4.3.1. Path

In path-based routing, the agent constructs complete routing paths for individual flows. This configuration typically relies on a global network view and centralized control, and is thus predominantly applied in SDN-based routing algorithms. Despite numerous paths between source and destination nodes, most are redundant or involve unnecessary back-and-forth, impairing the exploration efficiency. Therefore, researchers often predefine candidates based on the shortest path to expedite the learning process, as demonstrated in [46,54]. However, restricting candidates to the shortest paths may limit the agent's exploration capacity. Moreover, considering that hundreds of concurrent flows commonly exist in real networks, the decision space expands exponentially with flow scales, resulting in computational infeasibility, particularly in single-agent systems.

4.3.2. Link weights

Unlike explicit paths, link weights are assigned as cost metrics and intermediate references for subsequent metric-based routing decisions. As shown in [20,41,43], this action space is frequently employed in SDN-based algorithms, where the controller translates inferred link weights into updated forwarding tables for installation on forwarding devices.

4.3.3. Next hop

In contrast to path or link weight-based approaches relying on centralized control, hop-by-hop routing exhibits inherent scalability by enabling each node to autonomously determine its next hop. The size of the action space corresponds to the number of neighboring nodes, making it particularly suitable for MADRL-based routing algorithms, as adopted in [38,53,65].

4.3.4. Split ratios

Path, link weight, and next-hop-based action spaces generally target single-path routing. Although straightforward to implement, single-path

routing often results in traffic imbalance and limited fault tolerance. In contrast, multi-path routing distributes traffic across multiple available links, promoting load balancing. This action definition is compatible with both single-agent and multi-agent algorithms, either by distributing flows across candidate shortest paths (e.g., [37,60,66]) or among next-hop neighbors (e.g., [61–64]). The former represents an extension of path-based action spaces. The latter offers finer-grained control and greater randomness for traffic distribution, albeit at the cost of increased convergence time and computational complexity.

4.4. Reward

The reward serves as quantified feedback from the environment based on the latest policy inference. The temporal trend of rewards across consecutive time steps reflects the effectiveness of the current policy and substantially influences policy evaluation. Consequently, defining an appropriate reward function is critical for guiding desirable agent behavior.

A straightforward approach to reward shaping is aligning the reward function with the anticipated objective. In most cases, the primary objective is delay minimization. Accordingly, several studies [20,41–43] define the reward as the negative value of network delay. Similarly, works such as [61–63] focus on load balancing and define the reward as the negative value of network-wide maximum link utilization (MLU). To accommodate multi-objective optimization, the work in [46] proposes a multi-variate utility function, where the reward is calculated through a weighted summation of path-level available bandwidth, delay, and packet loss rate. Focusing on session-level utility, the work in [60] defines the reward as the aggregated utilities of all communication sessions, which are evaluated based on throughput and delay using the α -fairness model.

However, rewards derived from global network performance metrics necessitate additional data collection processes in MADRL systems, resulting in extra resource consumption, as employed in [61–63]. Besides, the global-sharing rewards may inadvertently encourage lazy agents by failing to differentiate individual contributions. To address these concerns, several MADRL-based routing studies, such as [59,65], advocate for individual rewards based solely on the traffic transmitted by the respective agent. Specifically, in these two works, the reward is defined as $r_n = q_n + l_n$, where q_n denotes the queuing time at agent n , and l_n represents the transmission delay to the next hop. Although this reward structure aligns with the decentralized nature of MADRL, the exclusive focus on individual rewards can induce selfish behaviors among agents, which may impair overall network performance.

As a compromise, the work in [37], in the context of a multi-domain network scenario, integrates average rewards from neighboring domains into the individual reward formulation while defining individual

rewards as $1 - MLU$, for trade-off between individual optimization and collective network performance. Furthermore, the work in [64] explores several reward compositions, introducing three meta-rewards: *ALL* for network-wide MLU, *DIRECT* for the MLU of directly connected links, and *BASIN* for the MLU of links reachable by the agent's routing decisions, as well as mixed and adaptive rewards by combining these meta-rewards through linear and adaptive weighting summation. Experimental results demonstrate that while meta-rewards alone may yield suboptimal policies, mixed reward strategies facilitate improved policy learning, and adaptive rewards, despite their complexity and need for meticulous weight tuning, achieve superior performance. This work highlights the profound impact of reward design on routing algorithm effectiveness and encourages a re-examination of conventional reward engineering practices.

Summary: This section reviews the common design of key elements in DRL-based routing algorithms, including agents, states, actions, and rewards. Each supports diverse customization schemes with distinct advantages. Generally speaking, centralized agents enable unified control, while distributed agents offer greater scalability; a univocal state is straightforward, whereas multi-dimensional representations provide richer environmental awareness; action spaces based on paths or link weights offer structural consistency, while those defined by next hops or split ratios are more scalable; global rewards promote full cooperation, local rewards help agents identify their individual contributions, and neighborhood-based rewards serve as a compromise. In practice, the design of these elements should be tailored to the characteristics of the managed network and the specific optimization objectives.

5. Model design

Model design constitutes a critical stage in the development life cycle, advancing algorithmic performance across multiple dimensions. Five main groups were found in the reviewed papers: performance enhancement, communication overhead reduction, decision complexity reduction, service continuity guarantee, robustness and generalization enhancement. This section elaborates on these considerations and examines notable strategies proposed in existing studies, as depicted in Fig. 5.

5.1. Performance enhancement

Performance enhancement is always the research focus. Recent works increasingly employ feature extraction techniques to fully unlock the potential of DRL-based methodologies, which provide agents with more accurate descriptors of network status. The reviewed literature identifies three primary feature extraction approaches, each designed for a specific feature type: statistical-based, time-series-based, and graph-structured.

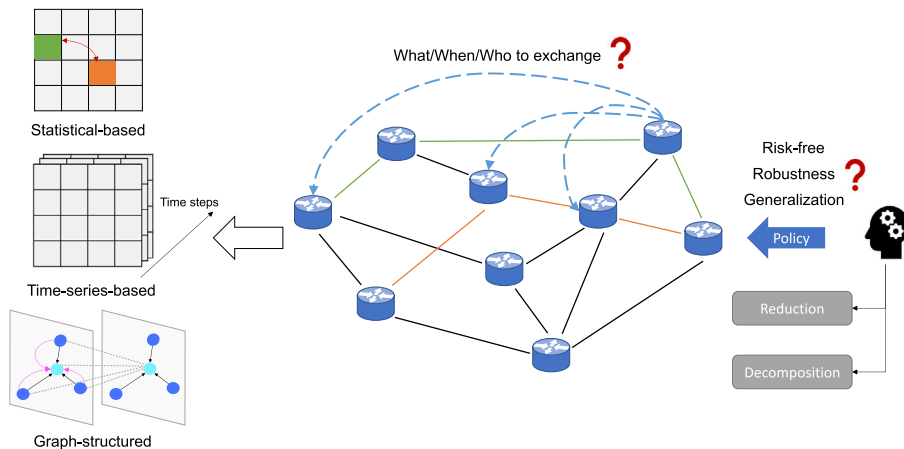


Fig. 5. Summary of considerations, issues, challenges in model design stage.

5.1.1. Statistical-based features

Features extracted from packet flows are predominantly statistical, as most network interfaces and management protocols adopt sampling and aggregation mechanisms. To enhance dynamic packet routing, several studies, such as those in [42,45], integrate convolutional neural networks (CNNs) with DRL agents to capture the interdependencies among coexisting flows. Building on a more customized network view, the work in [47] applies CNNs to process complex multi-dimensional data in SDN-IoT environments, including packet-in message frequencies, flow table occupancy rates, and channel occupancy between switch nodes and controllers, to accurately detect malicious activities. Empirical results in [42,45,47] demonstrate that CNN-augmented agents achieve superior QoS compared to vanilla agents. Overall, CNN-powered statistical feature extraction has gained widespread adoption due to its computational efficiency, offering a practical advantage in resource-constrained networks requiring rapid decision-making.

5.1.2. Time-series-based features

Time-series data, comprising event sequences organized in chronological order, naturally characterizes packet routing, which is inherently event-driven by session initiations and forwarding actions. Recurrent neural networks (RNNs), with their inherent capability to model sequential dependencies, are thus well-suited for such features. For instance, building on the findings in [75], which identified temporal correlations in periodic origin-destination (OD) flows, the work in [43] employs a basic RNN to capture traffic time-sequential characteristics for routing policy optimization, where the state space is represented as a matrix $s^{t \times n}$, with t denoting sampling times and n indicating the number of routers. Nevertheless, the proposed algorithm requires further fine-tuning due to its unstable convergence. Moreover, the work in [65] explores temporal correlations among forwarding actions, hypothesizing that learning to avoid repeatedly selecting the same routers could mitigate congestion. Accordingly, the state space includes the actions of the most recent packets and the destination nodes of upcoming packets queued for transmission. Experimental results demonstrate that LSTM-augmented agents substantially outperform traditional Q-routing [14] and backpressure [76] algorithms in reducing average packet delivery time across diverse traffic scenarios and topologies. Despite these advances, effectively capturing time-series features often necessitates a broad time window, which may compromise the agent's responsiveness to rapidly changing network conditions.

5.1.3. Graph-structured features

The intrinsic structure of the Internet naturally supports graph-based abstractions, enabling the application of a variety of graph representation learning techniques, such as graph embeddings and graph neural networks (GNNs) (e.g., message-passing neural networks (MPNNs) [77], graph convolutional networks (GCNs) [78], graph attention networks (GATs) [79], and graph transformer networks [80]). For instance, the work in [72] employs DeepWalk [81], a seminal method for learning social representations of graph vertices, to perform graph embedding. By incorporating node-level embeddings into the agent's state space, the model enables agents to infer inter-entity relationships within the network topology, thereby facilitating the bypassing of bottleneck nodes. While such graph embedding methods are efficient and scalable, they fall short in flexibility and representational capacity compared to end-to-end trainable GNNs. Typically, GNNs encode network information into compact node-level feature vectors. To preserve topological connectivity within these embeddings, GNNs propagate, aggregate, and update node representations through message-passing operations [82]. Each message-passing iteration constitutes a GNN layer, and stacking multiple layers enables multi-hop information diffusion, fostering high-level node awareness. In this sense, the initial node representation critically influences feature extraction performance. For example, the work in [59] characterizes each node using a three-dimensional binary vector, indicating whether it corresponds to the current node, a neighboring node,

or the destination. In the work in [67], the feature of node i is defined as the i -th row of the traffic matrix, capturing its associated flow dynamics. A more comprehensive node feature is defined in the work in [54], where nodes monitor adjacent link attributes such as bandwidth utilization, delay, and packet loss rate, including both traffic demands and policy execution feedback. Alternatively, the works in [50,68,69] directly focus on link attributes. Empirical results in these studies consistently demonstrate that GNN-based feature extraction improves routing performance compared to naive fully connected networks. On the other hand, the study in [73] leverages GCNs to model encrypted network traffic for the effective identification of various types of malicious activity. Moreover, the decision-making process integrates Bayesian inference to yield more accurate estimations of environmental states for the GCN. Experimental results show that the proposed approach achieves stable convergence toward higher rewards, effectively mitigating the impact of adversarial attacks. However, the convergence of GNNs often demands substantial computational resources, necessitating a trade-off between optimization benefits and operational overhead in practical deployments.

In summary, the selection of feature extraction strategies must align with the specific optimization objectives and underlying research motivations. Given that statistical, time-series, and graph-structured features respectively capture network characteristics at different perspectives, hybrid feature extraction approaches represent a promising direction for future exploration.

5.2. Communication overhead reduction

Due to the curse of dimensionality and the challenges of collecting distributed network status in real-time, many researchers have argued that centralized controller paradigms may face significant difficulties in large-scale networks. Meanwhile, with advancements in distributed and parallel computing technologies, MADRL has emerged as a mainstream research direction within the community. Consequently, many researchers have begun to introduce MADRL into routing optimization issues. However, MADRL routing scenarios often suffer from partial observability and non-stationarity, as limited sensor precision and perceptual aliasing may restrict agents to only immediate data from their forwarding devices. The conflict between individual policy inferences and collective system dynamics results in frequent failures. An intuitive solution is to incorporate information from other parallel agents. Nevertheless, any information exchange incurs increased communication overhead, which is undesirable in resource-constrained environments. To address this, our literature review explores three perspectives: what information to exchange, when to exchange it, and with whom to exchange it.

5.2.1. What information to exchange

Limited bandwidth restrictions are ubiquitous in practice. With the majority of bandwidth dedicated to data transmission, only a minimal portion remains available for information exchange. Therefore, directly exchanging raw information among agents is impractical in real-world applications. To this end, the work in [61] suggests that each agent encodes its local observation into a compressed representation using a deep neural network before interaction. This approach allows agents to cooperate with reduced communication overhead without compromising the quality of the exchanged information. Simulation results demonstrate that the proposed algorithm outperforms baselines in terms of convergence ratio and MLU.

5.2.2. When to exchange

The work in [63] identifies inefficiencies in continuous information exchange and designs a gated pruning mechanism to filter out unprofitable message exchanges. Specifically, the value of information is assessed by the difference in expected returns, denoted as $\Delta Q(o) = Q(o; a^C) - Q(o; a^I)$, where o represents the observation, a^C is the action inferred from communication, and a^I is the action inferred

independently. An individual shares local messages across the system only when the information value exceeds a predefined threshold T ; this also allows the agent to gain a global perspective from the centralized information coordinator for action inference. If the value does not exceed T , communication is withheld, and the agent acts on its own. In the initial stage of training, T is set to 1 (i.e., without message pruning) to facilitate the training of all components except the gating net. Once the model is well-trained, the actor network can generate a^C and a^I with $T = 1$ and $T = 0$, respectively, and the critic network estimates an approximately correct Q-value. Afterward, two methods for setting the threshold T are proposed: fixed and dynamic. For a fixed T , it can be calculated as $L_{\Delta Q(o)}[K \times T_m\%]$, where $L_{\Delta Q(o)}$ comprises Q-value differences for the latest K observations and $T_m\%$ is the desired information pruning ratio. It is important to note that $L_{\Delta Q(o)}$ is split by index instead of value, as $\Delta Q(o)$ typically has a non-uniform distribution. The advantage of a fixed T is that it ensures the actual number of pruned messages closely aligns with the desired $T_m\%$, even for large K . In contrast, the dynamic T is determined using an exponential moving average technique, represented as $T_t = (1 - \beta)T_{t-1} + \beta(Q(o_t, a_t^C) - Q(o_t, a_t^I))$, where β is a discounting coefficient for older T . This method allows $\Delta Q(o)$ to serve as an adaptive training label for the same observation o , enhancing adaptability in dynamic environments. Numerical results indicate that the gating mechanism can prune over 80 % of messages with minimal impact on performance, and a fixed $T_m\%$ can accurately control the message quantity. In comparison, this method significantly outperforms several state-of-the-art DRL-based and rule-based methods in terms of convergence ratio and MLU. This mechanism provides a principled approach for measuring information value, applicable across various fields with resource constraints to minimize unnecessary costs.

5.2.3. With whom to exchange

While most MADRL-based routing algorithms rely on neighborhood or network-wide information exchange, we advocate for a deeper examination of the scope of information exchange, driven by two key considerations. First, although broader information sharing enhances environmental awareness and cooperation, it raises scalability issues and privacy concerns in practical applications. Second, while a uniform global reward facilitates system-level optimization, it risks encouraging lazy behaviors by not differentiating individual contributions. In this context, the work in [38] systematically evaluates the performance of observations across various communication ranges, including global, 3-hop, 2-hop, 1-hop, and local node sets. Numerical results reveal that agents using 3-hop neighborhood information can achieve over 90 % of the theoretical performance limit in a network of 131 nodes. This evidence illustrates the law of diminishing marginal utility, whereby the incremental value derived from each additional unit of neighborhood information decreases once the acquired information has reached a certain threshold. Therefore, efforts should focus on determining the optimal scope of information exchange to enable effective decision-making with minimal communication overhead.

5.3. Decision complexity reduction

Decision complexity, primarily influenced by the size of the action space, constitutes a critical consideration in model design. When confronted with an extensive search space, DRL models may encounter significant challenges, including increased computational burdens, degraded performance, and the necessity for deeper neural networks and high-performance computing resources. Such requirements are particularly unfavorable in resource-constrained environments, where hardware can only accommodate light computational demands. As identified in [83–85], these issues represent major obstacles to deploying machine learning models as online processing tools on practical devices. Besides, Internet communications often occur within milliseconds for session establishment, termination, and data transfer [51]. Thus, routing policy inference must be completed rapidly to enable timely actions.

However, complex neural network architectures typically require longer computation times, negatively impacting responsiveness and user experience. Overall, reducing the action space is a widely adopted strategy to achieve moderate hardware requirements and faster response times.

One intuitive approach involves filtering low-efficiency forwarding operations that do not necessitate DRL-based management. A straightforward method is to constrain the action space to a set of paths computed based on the shortest path algorithms. Beyond this, the work in [37] proposes that agents exclusively manage elephant flows (high-traffic demands) while delegating mice flows (low-traffic demands) to traditional shortest-path routing. Simulation results demonstrate that this semi-participatory DRL-based routing algorithm enables efficient training with satisfactory performance outcomes.

Another strategy to alleviate decision complexity is to partition the original action space. For instance, the work in [38] introduces a sequential operation through a multiple-inference mechanism [86]. In the modeled inter-domain routing scenario, agents within each autonomous system must simultaneously route multiple co-existing flows. However, the candidate next-hop sets vary among flows due to BGP route advertisements. Treating the action space as the combination of these next-hop sets would result in an extremely large search space. By adopting multiple inferences, the agent generates actions for each flow sequentially, thereby simplifying each decision to the neighboring set, whose size is determined by the agent's degree. Similarly, the work in [54] addresses a scenario where a centralized agent must select paths for flow requests in different network slices. Since candidate path sets differ among slices, the full action space becomes the product of individual slice options, severely complicating training. To address this, a parallel approach is designed by integrating multi-task learning into the DRL model, where the output layer is divided into multiple branches, each responsible for a single slice, thus reducing the action space size to the sum of the path sets across slices. In summary, the sequential approach reduces decision complexity per inference but increases delay. The parallel approach enables faster responses but requires more hardware for its complex architecture. The ultimate choice between sequential and parallel designs depends on practical resource constraints and application-specific priorities.

5.4. Service continuity guarantee

The performance of production networks is closely tied to customer satisfaction. Nowadays, users demand high-speed data transmission and uninterrupted service availability. Even minor service disruptions can result in customer dissatisfaction, leading to churn, negative brand perception, and financial losses for service providers. Therefore, the extensive network of interconnected infrastructures, such as routers and switches, must maintain high operational reliability to ensure network uptime.

In this sense, DRL-based routing solutions may face two principal challenges. First, hop-by-hop and multi-path routing models inherently introduce forwarding uncertainty, heightening the risk of routing loops. In such scenarios, packets may circulate indefinitely without reaching their destination, resulting in prolonged transmission delays and unnecessary resource consumption. To address this issue, the works in [37,66] pre-compute loop-free path candidates using a traffic-oblivious selection algorithm [87] and a directed acyclic graph (DAG), respectively.

Second, inherent biases in deep neural networks may lead to deviations from established routing policies, leading to random or suboptimal actions. The work in [53] addresses this challenge by incorporating a safe learning mechanism [88] into the DRL framework. Specifically, a validation module assesses the feasibility of generated paths, ensuring that only loop-free and congestion-free routes are immediately deployed. If validation fails, a fallback policy is triggered, relying on traditional methods such as shortest-path routing to determine the forwarding decision. This design guarantees that all implemented routes remain loop-free even under burst traffic and link failures. Simulation results

demonstrate that the safe learning mechanism accelerates online adaptation while mitigating the risk of severe delay spikes. Although effective, it is important to acknowledge that such validation mechanisms introduce additional system complexity. Alternatively, developing strategies capable of seamlessly integrating domain-specific traditional knowledge into DRL models could offer a promising direction for enhancing the reliability of generated forwarding actions without imposing significant overhead.

5.5. Robustness and generalization enhancement

As modern networks continue to scale and grow increasingly complex, link failures have become a routine occurrence in both wide area networks (WANs) and data center networks (DCNs) [89]. Due to the combinatorial nature of failures, the number of possible failure scenarios grows exponentially with network size. If the agent is unable to accommodate link failures, the model must be continuously retrained whenever a new failure occurs, posing two critical challenges: (i) initiating retraining directly in production environments is infeasible due to the unpredictable exploration inherent to DRL; and (ii) retraining incurs substantial time and computational overhead, leading to prolonged periods of suboptimal network performance. To address these issues, DRL agents must exhibit prompt adaptability to network failures, emphasizing overall robustness. However, prevailing DRL-based routing models face significant limitations in applicability, largely due to their reliance on global-level representations. In these models, the dimensionality of the deep neural network is coupled to the training topology, severely constraining generalization to environments with different scales or connectivity patterns.

One promising direction involves using graph neural networks (GNNs), which naturally support both global graph-level and element-level outputs [90]. For instance, works such as [54,59] adopt global graph-level inference, where the states of all elements are aggregated into a global representation before prediction. Although effective within fixed topologies, this approach preserves the model-topology coupling. In contrast, element-level inference applies the output function individually to each node or link, thereby decoupling the decision-making process from the network structure and enabling broader generalization. For example, the work in [67] proposes an element-level routing strategy that derives link weights from the concatenated features of the interconnected nodes. Simulation results show that the algorithm outperforms benchmarks in terms of end-to-end delay under unseen link disconnections. Nevertheless, this evaluation is limited to simple single-source, single-destination scenarios with modest link failures, making it difficult to extrapolate performance under more complex or large-scale failure events. Fundamentally, DRL is an experience-driven paradigm. However, exhaustive simulation of all possible failure scenarios during training is neither financially nor operationally feasible for laboratories or commercial operators. Designing robust DRL algorithms capable of operating reliably across a wide range of unforeseen failures remains a formidable challenge.

Beyond link failures, safety, security, and scalability constraints prevent full-scale training in practical settings. Therefore, DRL models must generalize well across diverse, unseen networks. Several studies [68,69] explore the GNN-based element-level mappings to facilitate such generalization. Unlike the work in [67], which relies on node feature concatenation, these works employ MPNNs to process link-level features. However, both methods exhibit limitations. The model in [69] achieves performance comparable to OSPF on unseen graphs but does not demonstrate clear advantages. Similarly, the algorithm in [68] shows improved MLU over OSPF across different topologies but underperforms relative to ROAR [91], a DDPG-based algorithm retrained for each specific topology. Moreover, the topologies used in these evaluations, i.e., Abilene (12 nodes, 30 links), Cernet (14 nodes, 32 links), and GBN (17 nodes, 52 links), are within the scale range of the training topologies (e.g., NSFNET with 14 nodes and 42 links, GEANT2 with 24 nodes and 74 links). Generalization to substantially larger or more

complex networks remains unverified. In contrast, the work in [74], which also employs element-level graph reinforcement learning, does not directly apply the elementary MPNNs to process link features. Instead, it proposes a fully link-centric message passing paradigm by first constructing a graph where links serve as the vertices, and then performing feature extraction using an advanced GraphTransformer network [80]. Experimental results demonstrate that this link-as-vertex strategy yields significantly better performance than node-centric message propagation. Moreover, the proposed algorithm generalizes well not only on the training topology but also on unseen topologies that are smaller, larger, denser, or substantially more complex, while exhibiting strong robustness against link failures.

Summary: This section focuses on the most critical stage in the development life cycle, presenting solutions to five key challenges. Across all studies, learning efficiency remains a central concern. To this end, researchers typically employ statistical, temporal, and graph-based feature extraction methods to fully exploit the agent's potential. Given the resource constraints of real-world environments, efforts have been made to reduce communication overhead from three dimensions: message content, frequency, and interaction entities. To further reduce costs and response delays, decision space reduction and decomposition techniques are adopted, either by eliminating redundant decisions or by transforming them into sequential/parallel decision-making tasks. More importantly, to ensure a smooth user experience, additional mechanisms are introduced to enforce loop-free or harmless routing paths. From a long-term perspective, GNNs have been integrated to improve robustness and generalization, thereby mitigating the need for costly retraining in the face of link failures or topology shifts.

6. Model training

After completing the algorithm design, researchers perform interactions between agents and environments to guide the agents toward convergence to optimal routing policies, a process known as model training. In this section, we first introduce the common DRL training paradigms used in this field. In addition, given that the training process typically demands substantial computational resources, which poses a significant burden for most research institutions, we further analyze how to reduce training costs from two perspectives: minimizing the number of iterations required during training and reducing the total number of trials needed to reach an optimal solution.

6.1. Common training paradigms

The training procedure for a single agent typically follows the fundamental DRL interaction loop (as shown in Fig. 2). In contrast, training MADRL models has been a computational challenge since the early 2000s [92], primarily due to the complex interactions among agents. The implementation of MADRL-based algorithms is generally organized into two stages, i.e., training and execution, and mainly follows two paradigms. The first is centralized training with distributed execution (CTDE). During centralized training, parallel agents share the same network parameters, which are updated simultaneously using the experiences collected from all agents. During distributed execution, each agent independently infers forwarding decisions based on its observations. The second paradigm is distributed training with distributed execution (DTDE), where each agent independently develops its policy and infers actions without centralized coordination. Both paradigms adopt distributed execution to provide devices with high autonomy. However, both centralized and distributed training face practical limitations. On the one hand, constrained bandwidth makes it difficult to gather empirical transitions from widely distributed forwarding devices and update network parameters synchronously. On the other hand, fully distributed learning without peer-to-peer perception often suffers from partial observability and non-stationarity, preventing efficient policy learning.

To address these issues, most existing MADRL-based routing studies adopt a decentralized training paradigm adapted to real-world conditions. In this paradigm, each agent maintains a separate neural network

with personalized parameters for policy updates and action inferences, and customized inter-agent communication mechanisms are introduced to enhance environmental perception. Actor-critic (AC) architectures that decouple policy updates and action inferences further enhance the applicability of such paradigms. In this context, well-informed critic networks can accelerate policy evolution during training but remain inactive during execution, enabling independent decisions without communication.

Furthermore, since routing optimization inherently aims to coordinate the traversal of coexisting packets across various forwarding nodes, several studies explore more collaborative training paradigms. Rather than relying on pre-defined routing algorithms, the work in [62] proposes that each agent can improve its training efficiency by learning and adapting to the policies of its teammates. The core assumption is that when all agents know each other's actions, the environment becomes stationary despite ongoing policy updates. Following this idea, Mao et al. [62] introduces a centralized critic network embedded with a multi-head attention mechanism to accurately model the evolving joint policy among agents. In this setting, the attention-assisted critic incorporates the observations and actions of all agents to compute personalized Q-values for each agent, enabling targeted policy updates. Benefiting from the decoupled structure of actor-critic architectures, agents require only local observations for action inference, thus avoiding excessive communication overhead during execution. Experimental results demonstrate improved learning efficiency and faster convergence compared to state-of-the-art MADRL algorithms, attributed to enhanced inter-agent cooperation.

Despite the accelerated convergence, Mao et al. [62] introduces additional resource consumption due to increased communication and the computational load imposed by the attention module. In response, the work in [59] explores cooperative training paradigms inspired by federated learning (FL) [93], which emphasize localized model training. Instead of exchanging raw information, FL periodically aggregates network parameters from individual agents via a centralized controller and broadcasts an updated global model, significantly reducing communication overhead while preserving user privacy. Besides, the authors modify conventional FL update rules by introducing asynchronous local updates, lowering computational demand for less active routers. In addition to global aggregation, a regional training scheme is proposed, which restricts parameter sharing to neighboring agents. This paradigm eliminates the need for a central controller and achieves comparable performance to global training. Compared with previous cooperation mechanisms, these FL-based paradigms achieve lower resource consumption, protect agent autonomy, and preserve privacy, serving as a promising direction for future research in scalable and efficient MADRL training.

6.2. Training iterations reduction

Due to limited prior knowledge and low sample efficiency, agents often require extensive trial-and-error iterations to achieve convergence, particularly in complex network environments. One solution identified in the reviewed papers is pre-training, enabling agents to acquire preliminary routing knowledge before formal training. For instance, the work in [53] employs a supervised learning paradigm to guide the agent in learning the shortest path principle within lightly loaded network scenarios. Similarly, the work in [59] utilizes off-policy techniques to pre-train neural networks using experience transitions derived from a well-trained Q-routing policy [14] under low network load. Simulation results indicate that increased pre-training steps significantly enhance model performance.

Alternatively, the work in [60] aims to enhance the sample efficiency during training. First, a traffic engineering (TE)-aware exploration strategy is introduced, where, with probability ϵ , the agent selects actions as $a_{\text{base}} + \epsilon \cdot \mathcal{N}$, and with probability $1 - \epsilon$, it selects actions as $a + \epsilon \cdot \mathcal{N}$. Here, a_{base} denotes a TE baseline solution (e.g., NUM or shortest path), a is the

actor network's output, and $\mathcal{N}$ represents uniformly distributed noise. As training progresses, ϵ gradually decays. This exploration strategy ensures that early-stage actions, which are often less informed, are biased toward traditional solutions, providing effective experience tuples that accelerate policy learning. Second, Xu et al. [60] extends the prioritized experience replay mechanism originally designed for DQN [94] to continuous control settings within the actor-critic framework. The priority metric integrates two components: the temporal-difference (TD) error associated with the critic network and the Q-gradient related to the actor network. Simulation studies demonstrate that these enhancements substantially mitigate the performance limitations of vanilla DDPG [95] in routing tasks, achieving superior end-to-end delay, throughput, network utility, and robustness compared to four baseline methods (shortest path, NUM, load balancing, and vanilla DDPG) across diverse network topologies. The work in [72] proposes another online guided learning process that incorporates intrinsic curiosity modules (ICM) [96] and an incremental training paradigm. Specifically, ICM generates an intrinsic reward signal on top of the extrinsic reward naturally received from the environment for more efficient exploration and learning. Noticing that routing algorithms struggle to converge to desired policies under limited training budgets in complex network environments, the authors adopt an incremental training strategy inspired by the pattern of human learning. During training, traffic demands and task complexity are gradually increased across stages. Initially, a single-flow transmission task is deployed in a lightly loaded environment, leading to a base model that acquires fundamental decision-making capabilities. Subsequently, the model is transferred to increasingly complex environments, where it is further refined with minimal additional training steps. This process yields an improved model that leverages prior knowledge to achieve superior decision-making efficiency in complex scenarios with reduced training overhead.

6.3. Optimization trials minimization

In this field, it is common to encounter various solutions employing different DRL paradigms. Given the wide array of DRL paradigms, selecting the most suitable one for routing optimization is crucial. Most works typically base their selection on the action type, with DQN managing discrete actions like paths and next hops, while DDPG is used for link weights and split ratios. After selecting a DRL paradigm, researchers often need to test multiple hyper-parameter settings to identify the configuration that yields the best performance. This process usually consumes time and energy that is often unforeseen. Specifically, each additional hyper-parameter increases the dimensionality of the search space. Due to the continuous nature of hyper-parameters and limited problem-specific knowledge, defining the boundaries of the search space becomes challenging. In this context, while there are three main types of DRL paradigms, value-based, policy-based, and actor-critic-based, each possessing numerous variants, it is advisable to consider paradigms that are less sensitive to hyper-parameter values to alleviate the challenges associated with hyper-parameter optimization, such as proximal policy optimization (PPO) [97], soft actor-critic (SAC) [98]. For more detailed introductions to different DRL paradigms, please refer to [18].

Summary: This section first introduces the prevailing DRL-based training paradigms for routing, which are predominantly collaborative due to the inherent cooperative nature of routing tasks. Second, off-line pre-training and online guided learning are presented as effective means to mitigate prolonged early-stage exploration caused by the lack of prior knowledge. Finally, the principles for selecting appropriate DRL paradigms are highlighted to avoid the costly trial-and-error process of hyperparameter tuning.

7. Model verification

Model verification is a critical stage prior to final deployment, responsible for comprehensively evaluating model performance. Although a gap exists between laboratory environments and production networks,

simulations should strive to closely replicate the complexity and dynamics of real-world networks to provide a reliable performance evaluation. This section addresses issues related to the three steps in the model verification process: environment setup, requirement decoding, and formal verification.

7.1. Environment setup

7.1.1. Software platform

A common approach for creating simulation environments is to utilize deep learning frameworks such as TensorFlow and PyTorch. However, for precise modeling of packet-level routing dynamics, discrete event simulators like OMNET++ [99] and NS3 [100] have become the mainstream tools within the research community. Both simulators are modular and extensible, with Python-supported interfaces, offering a high degree of customization. In contrast, OMNET++ emphasizes component-based modeling, while NS3 adopts a more integration modeling. Note that the rich feature sets of these simulators often entail a steep learning curve and increased implementation complexity. Therefore, it is critical to strike a balance between simulation fidelity and development overhead when selecting a platform. For SDN-specific simulation tasks, where the separation of control and data planes is fundamental, Mininet [101] has emerged as a widely recommended open-source platform. It is compatible with OpenFlow [102] and enables the emulation of real kernel, switch, and application code on a single machine, e.g., virtual machine, cloud instance, or native system, through command-line configurations. Via northbound APIs, Mininet supports seamless integration with a variety of SDN controllers, including OpenDaylight (ODL) [103], Floodlight [104], Ryu [105], and PoX [106]. Among these, ODL and Floodlight are generally favored for constructing scalable, production-grade environments, whereas Ryu and PoX prioritize modularity and ease of use, making them well-suited for lightweight experimentation and rapid prototyping.

7.1.2. Topology formation

The network topologies adopted in the reviewed studies generally follow two main trends: artificially synthesized topologies and empirically derived real-world topologies. Among the synthetic approaches, two widely used toolkits have emerged. The first is the Barabási-Albert (BA) algorithm [107], which generates scale-free networks by capturing two fundamental characteristics observed in real-world topologies: growth and preferential attachment. The second is the Boston University Representative Internet Topology Generator (BRITe) [108], notable for its flexibility, universality, and user-friendly interface. While synthetic topologies enable controlled experimentation with customizable parameters, real-world topologies offer more authentic and practical testbeds, particularly for evaluating algorithms intended for deployment in operational networks. Currently, extensive datasets have been curated by Rocketfuel [109] and the Internet Topology Zoo [110]. Commonly adopted topologies include OS3E, NSFNET, GEANT, Abilene, Sprintlink, ARPANET, AT&T, EBONE, Tiscali, T-Lex, HARNET, BTN, GBN, CERNET, and Savvis, based on the reviewed literature.

7.1.3. Traffic generation

Traffic dynamics are fundamental characteristics of real-world networks, yet access to authentic network data is often constrained due to commercial sensitivities. Nevertheless, several open datasets are available for research purposes, such as those from Abilene, GEANT, Tor [111], and the Physics Experiment Data Export (PhEDEx) project [112]. Model-based and flow-based traffic generation are alternative solutions. Specifically, model-based approaches for traffic generation aim to reproduce the statistical properties of realistic traffic using mathematical models like normal distribution, Poisson processes, and the gravity model, which can be implemented by the Python library TMGen [113]. In contrast, flow-based generation seeks to replicate the content and arrival times of flows similar to real traffic conditions by copying or modeling real interactions through scripts. Some flow-based generation tools

include BRUTE [114], iPerf3 [115], and Ostinato [116]. Specifically, BRUTE is designed to generate Ethernet traffic for either IPv4 or IPv6. iPerf3 can generate traffic for TCP, UDP, and SCTP protocols via interactions between clients and servers. The Ostinato tool can generate traffic with different protocols at varying rates.

7.2. Requirement decoding

Accurately decoding domain-specific commercial requirements into verifiable performance metrics is a prerequisite of verification activities. In network routing optimization, elementary functional requirements include improved delay, throughput, packet loss rate, and maximum link utilization. To resonate with engineering and business perspectives, we further identify five practicality-oriented performance metrics, including:

7.2.1. Reproducibility

Reproducibility has emerged as a high-profile concern in recent years, marked by frequent failures in replicating published results [117]. To enhance the industry's confidence in academic research, offering perfectly annotated open-source code is recommended [118].

7.2.2. Cost efficiency

Cost-effectiveness is a primary concern for ISPs. In this regard, researchers should detail aspects such as running time (Internet communications normally take place in milliseconds to transfer data), computational requirements (CPU and GPU), and storage needs (RAM). On this basis, ISPs can weigh the performance advantages and required budgets, enabling a comprehensive evaluation of economic benefits.

7.2.3. Robustness

Real-world networks are prone to various internal and external disruptions, such as link congestion, hardware failures, and natural disasters. Although it remains infeasible to exhaustively test all potential failure scenarios, DRL-based routing algorithms should demonstrate reasonable handling of edge cases and overall robustness.

7.2.4. Generalization performance

Given the impracticality of conducting full-scale training and testing in real-world networks, models should generalize well to previously unseen data in order to establish credibility for practical deployment. In this sense, verifications should occur under network conditions with unfamiliar traffic patterns and topological structures, where the model is expected to demonstrate consistently superior performance.

7.2.5. Partial deployability

Given the complexities in software integration and financial constraints, assessing the performance gains achievable through partial device upgrades is advisable. This evaluation can provide a factual basis for the subsequent incremental deployment approach that facilitates a phased implementation with minimized operational risks.

7.3. Formal verification

Each proposed algorithm should undergo comprehensive evaluation against benchmark methods across a range of performance indicators. In the following discussion, we first present the performance evaluation reported in the reviewed literature based on general metrics such as end-to-end delay, packet loss rate, and throughput. Subsequently, we examine evaluations of practicality-oriented indicators. To avoid redundancy, experimental results that have been discussed in other sections are not repeated here.

7.3.1. Evaluation of general performance metrics

The works in [61–64] share similar verification processes, utilizing convergence ratio (CR) and maximum link utilization (MLU) as the primary performance indicators. In the initial attempt [61], the evaluation proceeds on simplistic topologies featuring only two/five ingress-egress

router pairs. The proposed algorithm consistently outperforms baseline MADRL-based algorithms following DTDE and CTDE paradigms. To improve authenticity, the follow-up research [62] adopts a 12-node application topology. Numerical results demonstrate that the proposed algorithm not only achieves lower MLU compared to state-of-the-art MADRL-based methods (MADDPG [119] and PSMAADDPGV2 [120]) and the rule-based routing method WCMP [121] but also exhibits even more exceptional performance as the network scale expands. In the work [63] focusing on intermittent information exchange, the proposed algorithm additionally undergoes specialized evaluation towards the precision in information removal and performance sustainment capability. In another specialized investigation centering on the reward-sharing range [64], the authors conducted a comparative analysis of CR and MLU for seven reward compositions across three different network scales. In conclusion, while all meta-rewards generate suboptimal policies, mixed reward signals are readily applicable for learning improved policies, and adaptive rewards achieve optimal results at the cost of careful weight adjustment.

The work in [41] employs the real-world Sprint network topology, and utilize the gravity model to generate 250 TMs for each of the four traffic intensity levels, ranging from 10 % to 100 % of the total network capacity. Experimentally, the well-trained agent achieves lower network delay than random routing configurations and OSPF, and illustrates superior throughput compared with two existing throughput optimal solutions, i.e., SDN-LB [122] and QAR [123].

The work in [60] validates the algorithm performance on two real-world network topologies, i.e., NSFNET and ARPANET, and a BRITE-generated topology. Each network is assigned twenty communication sessions, whose packet arrivals follow the Poisson process with mean values ranging from 10 Mbps to 40 Mbps. Simulation results demonstrate that the proposed algorithm outperforms the shortest path, load balancing, NUM, and DDPG in terms of network delay, throughput, and utility.

The work in [59] adopts similar setups, using altered topologies such as an irregular 6×6 grid network and a real AT&T North America network. On this basis, the proposed algorithm is compared with five baseline methods, including the shortest path, Q-routing [14], Hybrid [124], DQN-routing [125], and Global Routing (i.e., an instantaneous optimal routing) under nine network load levels approximately ranging from 0 to 4. Extensive results indicate that the proposed algorithm achieves a low delay that is only second to Global Routing. Through tabular analysis of the mean value and variance of delay across three learning paradigms, the cooperative learning paradigm achieves more desirable and stable network performance than centralized learning and federated learning.

The work in [54] uses three practical network topologies obtained from the Internet Topology Zoo and the real-world traffic dataset, Tor, for traffic generation. Simulation results reveal that the proposed algorithm outperforms five baseline solutions (ACKTR, dueling DQN, GCN without DiffPool, shortest path with hard slicing, and load balancing with hard slicing) across three networks concerning delay, throughput, packet loss rate, and service level agreement (SLA) satisfaction ratio.

7.3.2. Evaluation of practicality-oriented indicators

The first DRL-based routing solution [20] is evaluated on a scale-free topology comprising 14 nodes and 21 full-duplex links, covering ten traffic intensity levels ranging from 12.5 % to 125 % of the total network capacity, each involving 100 TMs generated by the gravity model. Simulation results reveal that the well-trained DRL agent outperforms random routing in terms of network delay. As a pioneering research, all datasets and source code used in this work are openly accessible at [126], aiming to inspire subsequent academic research.

The work in [43] establishes the testbed OS3E topology using physical PICA8 switches and runs customized scripts to imitate real traffic patterns featuring periodic and random flows. In comparison with the shortest path across varying random flow ratios and traffic intensities,

the proposed algorithm demonstrates superior delay and packet loss rates. Specifically, the performance advantage gradually decreases with added random flows but increases with growing traffic intensities. In addition, the authors measured the running time of the proposed algorithm and the shortest path across different network scales. Despite a prolonged running time with increasing nodes, the proposed algorithm consistently makes millisecond-level decisions and outperforms the shortest path.

The evaluation of [46] primarily revolves around a 23-node GEANT topology and a 48-node topology generated by the BA algorithm. While GEANT features a public traffic dataset, the traffic within the synthetic topology is generated by TMGen. Evaluation metrics encompass traditional performance metrics, i.e., throughput, loss ratio, and delay, as well as a path-state indicator known as stretch, implying the comparison between the generated path length and the theoretical shortest path. Simulation results show that the proposed algorithm exhibits superior delay, throughput, and loss rate at the expense of a slightly higher stretch, compared with the Dijkstra algorithm and its variants. Besides, the authors examined response time to topology changes, along with the CPU and storage memory consumption. The DRSIR prototype and all test scripts are available in [127].

In the robustness-specialized study [67], the authors preserved the primitive trained algorithm model and tested its performance in the topology whose edges are randomly removed. Simulation results show that the proposed algorithm consistently exhibits the lowest delay and packet loss compared to random routing, OSPF, PPO, and GDDR [69]. Furthermore, they applied the agent trained with a 6.5 Mbps flow rate to a network where the flow rate gradually increased from 1 Mbps to 6.5 Mbps. According to the simulation, all algorithms maintain low delay at low flow rates. Nevertheless, only the proposed algorithm sustains satisfying performance throughout the changing process and showcases stretched advantage as the rate increases. For enhanced credibility, the same experiment is also conducted in a real-world network topology, Abilene, which reaffirms the algorithm's performance.

The work in [37] calculates a customized performance indicator by dividing the observed maximum edge utilization by that under globally optimal routing, termed normalized congestion. Across two real-world topologies (i.e., Telstra from the Rocketfuel project and a network from Google Cloud [128]) and a large-scale synthetic topology generated by BRITE, the proposed algorithm limits congestion within 1.2 times the global optimal solution in over 90 % of tests. This performance significantly surpasses TRPO, hot potato routing (HPR) [129], and ECMP. In addition, the authors tested their robustness by randomly setting the link utilization of a link to 1, and compared their training speed. Simulation results indicate that the proposed algorithm exhibits superior robustness.

The work in [65] is initially evaluated on a 3×3 grid network, with a specific source-destination pair receiving a high fixed proportion of traffic. Testing results of well-trained models under various packet generation intervals and distribution ratios demonstrate that the proposed algorithm possesses greater adaptability and achieves the shortest average delivery time than Q-routing and Backpressure. Besides, the authors tested algorithms with abrupt and time-varying generation intervals for robustness evaluation. Simulation results show that the proposed algorithm accomplishes the lowest fluctuation and delivery time, thus capable of dealing with dynamic networks.

The initial DRL-based inter-domain routing algorithm [38] proceeds evaluations on a 131-node topology extracted from PhEDEx with 200 TMs, each containing 350 to 400 E2E flows. Flow-based routing simulation results show that agents equipped with 3-hop neighboring information achieve over 90 % of the upper-bound performance calculated by the mixed integer problem and approaches that of global observation. Even without control at the flow level, the proposed algorithm still yields improvements of up to 17 % in destination-based routing. To gauge scalability in large-scale networks, the authors further extended into a 10,000-node topology yet only equipped it with 20 DRL-driven nodes. Remarkably, the performance advantage remains at

16 % compared to default BGP and slightly lessens with fewer driven nodes, substantiating its deployability in real-world networks.

The work in [53] exemplifies one of the most comprehensive verification processes in current research efforts. The SDN architecture is implemented by Mininet and Ryu on a cloud-based server, modeling the 11-node Abilene and the 23-node GEANT networks with real-world traffic matrices and differentiated link capacities. In all network scenarios, the proposed algorithm consistently outperforms traditional shortest-path routing, load-balancing routing, and QoS routing regarding delay, throughput, and packet loss rate. To address practical considerations, the authors then tested the trained model under link failures and burst traffic, where the algorithm exhibits stable delay and throughput shortly after disturbances. Afterward, agents are partially deployed on key routing nodes adjacent to bottleneck links to assess the potential for incremental deployment. Surprisingly, such partial deployment achieves performance comparable to that of a fully deployed paradigm in the Abilene network; however, it proves less effective in the larger GEANT network, where many flows bypass the nodes controlled by the DRL agents, though it still outperforms the shortest-path method. Finally, researchers reported that each agent requires only 0.8 ms to select the next hop based on the input state. The source code is available at [130]. In summary, this verification addresses reproducibility, resource cost, robustness, partial deployability, and generalization performance concerning unfamiliar traffic patterns, providing a valuable reference for future research in this domain.

Summary: This section presents a comprehensive overview of the verification procedures for DRL-based routing algorithms, encompassing three key stages: environment setup, requirement decoding, and formal verification. In terms of environment configuration, we introduced widely adopted software platforms, topologies, and traffic generation schemes that closely approximate real-world conditions. Beyond conventional performance metrics such as delay, packet loss rate, jitter, and throughput, we further extracted five metrics that are critical for real-world applicability: reproducibility, cost efficiency, robustness, generalization performance, and partial deployability. Correspondingly, we provided verification methodologies for each of these metrics by representative research works, offering references for readers seeking to assess DRL-based routing models under practicality-oriented requirements.

8. Model deployment

Due to commercial constraints, most research works primarily present proof of concept rather than actual deployment. Even so, some studies offer valuable insights for deploying DRL-based routing solutions in real-world scenarios. Given the complexity of these solutions, which require significant development efforts, ISPs naturally expect their long-term maintenance. In this section, we address issues related to three critical steps in model deployment: integration, monitoring, and updating.

8.1. Integration

The integration step involves two main activities: building the infrastructure to support the model and deploying the model via software updates. While both activities primarily fall under systems engineering and are beyond the scope of this survey, our interest lies in determining the scope of technical upgrades. Despite being commonly adopted, the widespread deployment of intelligent capabilities across all network entities presents several challenges. First, minor software updates may strain legacy routers, requiring costly hardware upgrades. As indicated by the work in [131], upgrading all legacy routers with SDN switches encounters severe economic and technical issues. Second, the inherent stochasticity of neural networks, coupled with the disparity between laboratory simulations and real-world conditions, leads ISPs to distrust fully AI-driven routing systems, fearing potential service disruptions.

In response to these concerns, incremental deployment, implementing DRL-based routing algorithms on a limited subset of network devices,

offers a safer alternative that retains support from traditional network infrastructure. Many studies have shown that hybrid SDN can achieve network performance comparable to full-SDN enabled network, as noted by the work in [132]. In the field of DRL-based routing optimization, the work in [53] demonstrates that deploying trained agents on key routing nodes adjacent to bottleneck links achieves performance comparable to full deployment during the verification process. The work in [66] exclusively studies a hybrid SDN environment where only a subset of nodes is controlled by SDN using DRL-based routing strategies, while others remain legacy. These agents use a centralized critic for training but act independently online, with observations from NetFlow/sFlow monitored on SDN switches. During testing on the CERNET, GEANT, and Arnes networks, the SDN deployment ratios of the proposed algorithm are set to 0.2, 0.3, and 0.4, where it consistently achieves better MLU than OSPF, ROAR [91], and a general multi-agent DRL-based routing algorithm [37], even exhibiting more advantages in larger network topologies.

While previous works on incremental deployment device selection have relied on bottleneck positions [53] and random choices [66], the work in [70] proposes a heuristic method to select driver nodes in networks. Following the pinning control theory introduced in [133], driver nodes are defined as those that can significantly influence the operational dynamics of the entire network, and others are defined as follower nodes, which have minimal impact on network dynamics. The basic idea is to assign nodes different probabilities based on their degrees and then select driver nodes according to these probabilities. Specifically, a node with a degree of one is marked as a follower node, as controlling it only affects the paths of flows traversing it. Similarly, if the neighboring nodes of a follower node each have a degree of two, these nodes are also classified as followers. After one round of selection, the nodes identified as followers are removed from the node set, and the graph is updated. In the new graph, a node with a degree of one is again marked as a follower. Subsequently, the remaining nodes are assigned a probability value based on their degrees in the current graph. Simulation results indicate that the proposed algorithm outperforms existing DRL-based schemes [20,60] by reducing the average flow completion time. Interestingly, the results reveal that controlling more driver nodes without encountering convergence issues does not necessarily lead to improved routing performance. In alignment with the conclusions in [134], the authors acknowledged the absence of a theoretical method for optimally selecting driver nodes for a given topology. However, due to its cost-effectiveness, ease of implementation, and enhanced operational safety, such an incremental deployment approach is increasingly favored within the networking community.

8.2. Monitoring and updating for efficient retraining

A prominent challenge faced by many DRL-based algorithms is their vulnerability to becoming outdated in the face of dynamic network conditions, which can lead to unacceptable suboptimal performance in practice. Common solutions, such as regular retraining and continual learning [135], often introduce considerable computational and operational costs; however, failure to address these issues may lead to a gradual decline in model performance over time. This dilemma raises two critical questions: when should retraining be initiated, and how can it be executed effectively, closely tied to monitoring and updating, respectively.

To facilitate the timely detection of abnormal forwarding behaviors, researchers should continuously monitor key business-driven metrics, such as delay, throughput, and packet loss rate. Statistical methods can then be employed to analyze the sampled data and assess whether deviations from expected performance occur. The precise threshold for this deterioration can be defined based on the specific requirements of the application. Once deviations are detected, signal the need for retraining. Notably, while a high sampling frequency is effective for capturing abnormal behaviors, it can also be resource-intensive, necessitating a thorough trade-off analysis.

Regarding updating, the work in [136] highlights an underlying challenge: retraining routing agents without interrupting network services is computationally prohibitive and introduces safety risks, as policies may degrade during exploration. Drawing inspiration from cognitive networks proposed in [137], the work in [136] proposes a dual-loop structure involving an offline simulation loop and an online execution loop. In this setup, the offline simulation loop regularly captures traffic demands and performance data from the online loop to optimize agent policies. Once the offline agents converge, their policy parameters are transferred to the online agents, allowing them to adjust to network changes with pre-optimized policies, mitigating the risks of uncertain exploration. However, this approach incurs significant computational efforts. To reduce retraining costs, the work in [71] introduces a generative adversarial network (GAN) [138] framework. Specifically, this design incorporates a fully connected network as a feature encoder. Once network conditions alter, a domain process is activated. While the original feature encoder provides real samples, a zero-sum game takes place between a newly introduced feature encoder and a discriminator. Eventually, the new encoder is able to map unknown state vectors in the target network to domain-invariant features recognized by existing policy networks. This limits retraining to the feature extraction network while keeping the core policy networks

unchanged. Simulation results show that this GAN-based approach effectively speeds up the convergence of retraining under varied link states and topologies. Despite the promising results, a fully integrated monitoring and updating system has yet to be developed, demanding further investigation.

Summary: This section focuses on three key steps in the deployment stage: integration, monitoring, and updating. While the integration phase has already seen several promising research directions, such as incremental deployment for stable and efficient device, the monitoring and updating phases remain relatively underexplored. Nevertheless, the dual-loop cognitive network paradigm and GAN-based domain-adaptation methods hold potential to facilitate rapid policy updates without compromising user experience.

9. Discussion

This survey comprehensively examines representative DRL-based routing algorithms following the established development life cycle. The stage-wise highlights of reviewed works are summarized in Table 6. However, numerous challenges persist in real-world network systems. In this section, we will offer insights into potential directions for future research.

Table 6
Stage-wise highlights of reviewed research works.

Ref.	Model Design	Model Training	Model Verification	Model Deployment
[37]	Decision complexity reduction, service continuity guarantee	Decentralized	Robustness	Full deployment
[38]	Communication overhead reduction, decision complexity reduction	Decentralized	Partial deployability	Incremental deployment
[20]	—	Single-agent	Reproducibility	Full deployment
[41]	—	Single-agent	General metrics	Full deployment
[42]	CNN-based feature extraction	Single-agent	General metrics	Full deployment
[43]	RNN-based feature extraction	Single-agent	Resource cost	Full deployment
[45]	CNN-based feature extraction	Single-agent	General metrics	Full deployment
[46]	—	Single-agent	Reproducibility, resource cost	Full deployment
[47]	CNN-based feature extraction	Single-agent	General metrics	Full deployment
[50]	GNN-based feature extraction	Decentralized	Robustness	Full deployment
[53]	Service continuity guarantee	Training iterations reduction	Reproducibility, robustness, partial deployment	Incremental deployment
[54]	GNN-based feature extraction, decision complexity reduction	Single-agent	General metrics	Full deployment
[55]	GNN-based feature extraction	Single-agent	Generalization	Full deployment
[57]	—	Decentralized	General metrics	Full deployment
[59]	GNN-based feature extraction	Federate learning, training iterations reduction	General metrics	Full deployment
[60]	—	Training iterations reduction	General metrics	Full deployment
[61]	Communication overhead reduction	Decentralized	General metrics	Full deployment
[62]	—	Attention-based cooperative learning	General metrics	Full deployment
[63]	Communication overhead reduction	Decentralized	General metrics	Full deployment
[64]	—	Decentralized	General metrics	Full deployment
[65]	RNN-based feature extraction	Decentralized	Robustness and generalization	Full deployment
[67]	GNN-based feature extraction	Single-agent	Robustness	Full deployment
[66]	Service continuity guarantee	Decentralized	Partial deployment	Incremental deployment
[68]	GNN-based feature extraction	Single-agent	Generalization	Full deployment
[69]	GNN-based feature extraction	Single-agent	Generalization	Full deployment
[70]	—	Single-agent	Partial deployment	Incremental deployment
[71]	—	Single-agent	General metrics	Gan-based updating
[72]	Feature Extraction via Graph Embedding	Training iterations reduction	Robustness	Full deployment
[73]	GNN-based feature extraction	Single-agent	General metrics	Full deployment
[74]	GNN-based feature extraction	Single-agent	Robustness and generalization	Full deployment

9.1. Integrate MADRL-based routing algorithms with scalable SDN architectures to adapt to future expanding networks

While SDN excels in logically centralized management, the challenges posed by increasing flow demands and expanding network scales highlight the limitations of its single-point control nature. Consequently, three prevailing SDN architecture designs have emerged known for excellent scalability [139,140]: (i) Horizontally distributed controllers, each dedicatedly managing a subnetwork/domain; (ii) Vertically hierarchical design, where local controllers handle frequent events and a main “root” controller takes for non-local rare events; (iii) Hybrid approach, integrating both data and control planes in decision-making. These well-scalable architectures serve as opportune platforms for MADRL-based routing algorithms. Although the work in [53] explores an attempt at distributing agents to each forwarding device, it still maintains the centralized SDN controller to issue ultimate routing configurations. How to simultaneously harness scalable SDN controllers and MADRL paradigms remains an open issue, yet holds promising potential for addressing future even larger-scale networks.

9.2. Investigate model-based DRL paradigms to improve the training efficiency of DRL-based routing algorithms

DRL fundamentally has two learning paradigms: model-free and model-based. While model-free methods learn from online interactions without explicit knowledge of the underlying dynamics, model-based methods utilize explicit models of the environment to calculate state transitions and rewards. Owing to the challenge of mathematically modeling real-world networks, model-free DRL methods have gained popularity in the routing research community yet have been criticized for low sample utilization and prolonged convergence, which result in costly training expenses. In contrast, model-based methods can effectively remedy these deficiencies and further facilitate planning techniques to explore unfamiliar network situations. As a result, researchers have resumed investigations into model-based DRL methods. In addition to the work in [141] that benchmarks prevailing model-based DRL methods, the research in [142] devises two model-based DRL algorithms exclusively for multi-hop scheduling and routing of real-time flows, yielding promising empirical outcomes that inspire future research.

9.3. Address asynchronous and noisy observations that undermine the practical feasibility of theoretical DRL-based routing strategies

Most current research assumes that the network status collected from the environment is always up-to-date and accurate, with agents possessing synchronized and precise network observations. However, in practice, data collection can be extensive, unstable, and resource-intensive, especially in large-scale networks. For example, distributed SDN networks can only achieve partial inter-controller synchronization and operate under the eventual consistency model [143]. This raises several critical yet often overlooked questions: How can agents maintain synchronized network observations within constrained synchronization budgets to prevent performance degradation? How can agents optimize routing with asynchronous views, and what is the tolerance level for such discrepancies? To address the first issue, the work in [144] introduces a DRL-based view synchronization approach that outperforms the anti-entropy protocol used in the state-of-the-art ONOS SDN controller [145]. However, they relied on the traditional shortest path algorithm for inter-domain routing and left the latter two issues unresolved. Furthermore, industrial sensor devices frequently encounter noise interference and limited sensing accuracy, which can introduce cumulative errors that compromise training stability and the evolution of routing policies. While research on DRL-based routing under noisy conditions remains sparse, valuable insights can be drawn from DRL applications in autonomous driving and robotics [146,147], where agents effectively manage noise and uncertainty.

9.4. Enhance the interpretability of DRL-based routing algorithms

Interpretability is essential for building trust among the target audience [148,149]. Due to the involvement of large-scale infrastructures and requirements for uninterrupted service provision, ISPs require enhanced interpretability of routing algorithms to comprehend their internal principles, workflows, and potential risks. While research on the interpretability of RL has developed into a distinct subfield and attracted significant attention [150–152], advancements have yet to be introduced to the field of routing optimization. While interpretability remains a complex issue across all deep learning methods, a more feasible alternative is to integrate models with high credibility into the DRL decision-making process. For example, the work in [16] only utilizes agents to identify critical flows in networks that require rerouting. The ultimate rerouting configuration is formulated as a simple linear programming (LP) problem using solvers like Gurobi [9]. Extensive evaluations demonstrate that the proposed algorithm achieves near-optimal performance by rerouting only 10 % to 21.3 % of the total traffic. The structured nature of linear programming enhances the interpretability of the final routing actions, while the involvement of agents effectively reduces the computational burden on LP solvers, ensuring acceptable response times. This work represents a logical path in this field.

9.5. Manage the varied number of agents without full retraining

Beyond link failures, the addition of new nodes is another common cause of topology changes. While a straightforward solution would involve retraining all agents to adapt to the new network, a more resource-efficient approach focuses on knowledge generalization, which can be categorized into two types. The first type leverages internal historical experiences, such as horizontal transfer learning [153] and vertical federated learning [154]. For instance, the work in [155] presented an incremental learning approach to manage varying numbers of routing agents. When new agents are introduced, the generalized model, derived from the average parameters of existing agents, is transferred to the new agents while retaining the original models. Conversely, if the number of agents decreases, the updated model is shared with all remaining agents. The second type involves external knowledge sources, exemplified by teacher-student learning [156], where knowledge is transferred from a complex teacher model to a simpler student model. Although both approaches require fine-tuning for convergence, they significantly reduce resource costs compared to full retraining, making them advantageous for practical applications.

9.6. Establish consolidated domain specialized verification systems for valuable comparison

The current verification processes for DRL-based routing algorithms face two significant challenges. First, these algorithms are developed across diverse platforms, topologies, and traffic conditions, often with inconsistent performance indicators. This variation complicates meaningful performance comparisons on a standardized scale. Second, despite rigorous academic efforts, researchers are often unprepared for emergency or edge-case scenarios due to a lack of transparency regarding the operational realities in real-world networks. In light of these challenges, we call for the community to collaboratively establish and maintain community-wide, open verification systems. Specifically, these should include simulation environments that reflect the constraints and complexity of real-world network operations; modern TM datasets that replace outdated ones (e.g., Abilene, GEANT2), which no longer capture the dynamics of contemporary networks; and unified performance metrics with standard reference values to ensure consistency in evaluation. Such a verification system would enable meaningful and fair comparisons across proposed algorithms in terms of practicality-oriented metrics such as cost efficiency, robustness, and generalization capability. Moreover, it would ground algorithm design in practical, application-relevant objectives.

9.7. Scientifically select devices for incremental deployment

Current approaches to device selection in incremental deployment largely rely on heuristic methods. For instance, the work in [53] deploys trained agents on key routing nodes adjacent to bottleneck links, the work in [66] conducts SDN deployment on random choices, and the work in [70] proposes a heuristic method that relies on node degree. However, bottleneck-based methods are effective only in the current network environment, while degree-based methods are closely tied to the topology structure without adequately addressing traffic demands. Despite the likelihood of accidents in real-world networks, traffic in production networks typically exhibits periodic patterns, and link failures are often not as severe as they might appear. In this context, we suggest researchers develop a more scientific approach to device selection that considers both traffic-side and topology-side characteristics.

9.8. Leveraging generative diffusion models to enhance DRL in dynamic network routing optimization

Generative Artificial Intelligence (GenAI) has marked a significant milestone in the evolution of AI, capable of synthesizing entirely new data rather than merely analyzing or classifying existing datasets. Among the diverse array of GenAI techniques, Generative Diffusion Models (GDMs) have emerged as particularly promising, due to their superior data generation quality, architectural flexibility, and relative ease of implementation. Notably, the training process of the denoising network within GDMs, driven by external environmental feedback, inherently aligns with the DRL paradigm. Conversely, the intrinsic strengths of GDMs in modeling complex data distributions can significantly enhance DRL algorithms, particularly in networked environments characterized by dynamics and complexities. In this context, the work in [157] provides a comprehensive tutorial that explores the integration of GDMs and DRL for wireless network optimization, broadening the algorithmic toolkit available to researchers and practitioners. Later, the work in [158] represents an early and notable effort to apply a diffusion-based multi-agent DRL algorithm to the joint optimization of DNN partitioning, task offloading, and resource allocation in dynamic vehicular networks. The empirical results demonstrate clear advantages over conventional DRL baselines. Although the underlying system models in wireless and ISP networks differ, both scenarios share the inherent complexity and dynamics of modern communication environments. As a result, these more advanced and adaptable algorithms unlock new possibilities in network routing optimization equally, which deserve in-depth investigation in the future.

10. Conclusion

This paper provided a literature review of contemporary DRL-based routing optimization. Initially, we introduced an original life cycle for developing feasible DRL-based routing algorithms. Then, we systematically analyzed critical considerations in each stage and demonstrated existing state-of-the-art solutions through cutting-edge works. Finally, we elaborated on future research directions in light of the survey findings. The objective of this survey is to help and inspire relevant academic researchers to gain a deeper understanding of the development of DRL-based routing algorithms.

CRedit authorship contribution statement

Yuqian Song: Writing – review & editing, Writing – original draft, Methodology. **Mao Ni:** Funding acquisition, Writing – review & editing. **Jun Liu:** Writing – review & editing, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Jun Liu reports that financial support was provided by the National Natural Science Foundation of China under the Grant No. 62371057. If

there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] R. Li, Z. Zhao, X. Zhou, G. Ding, Y. Chen, Z. Wang, H. Zhang, Intelligent 5G: when cellular networks meet artificial intelligence, *IEEE Wirel. Commun.* 24 (5) (2017) 175–183.
- [2] W. Saad, M. Bennis, M. Chen, A vision of 6G wireless systems: applications, trends, technologies, and open research problems, *IEEE Netw.* 34 (3) (2019) 134–142.
- [3] G. Malkin, RIP Version 2. Technical Report, 1998.
- [4] J. Moy, OSPF Version 2. Technical Report, 1997.
- [5] H. Smit, Intermediate system to intermediate system (ISIS) extensions for traffic engineering (TE), *IETF RFC 3784* (2004).
- [6] M. Chiesa, G. Kindler, M. Schapira, Traffic engineering with equal-cost-multipath: an algorithmic perspective, *IEEE/ACM Trans. Netw.* 25 (2) (2016) 779–792.
- [7] M. Pizzutti, A.E. Schaeffer-Filho, Adaptive multipath routing based on hybrid data and control plane operation, in: *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, IEEE, 2019, pp. 730–738.
- [8] C.U. Manual, IBM ILOG CPLEX optimization studio, Version 12 (1987–2018) (1987) 1.
- [9] LLC Gurobi Optimization, Gurobi optimizer reference manual (2021).
- [10] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, A. Cabellos-Aparicio, RouteNet: leveraging graph neural networks for network modeling and optimization in SDN, *IEEE J. Sel. Areas Commun.* 38 (10) (2020) 2260–2270.
- [11] B. Mao, Z.M. Fadlullah, F. Tang, N. Kato, O. Akashi, T. Inoue, K. Mizutani, Routing or computing? the paradigm shift towards intelligent computer network packet transmission based on deep learning, *IEEE Trans. Comput.* 66 (11) (2017) 1946–1960.
- [12] R. Kirby, S. Godil, R. Roy, B. Catanzaro, CongestionNet: routing congestion prediction using deep graph neural networks, in: *2019 IFIP/IEEE 27th International Conference on Very Large Scale Integration (VLSI-SoC)*, IEEE, 2019, pp. 217–222.
- [13] B. Yan, Q. Liu, J. Shen, D. Liang, Flowlet-level multipath routing based on graph neural network in openflow-based SDN, *Future Gener. Comput. Syst.* 134 (2022) 140–153.
- [14] J. Boyan, M. Littman, Packet routing in dynamically changing networks: a reinforcement learning approach, *Adv. Neural Inf. Process. Syst.* 6 (1993).
- [15] Q. Xu, Y. Zhang, K. Wu, J. Wang, K. Lu, Evaluating and boosting reinforcement learning for intra-domain routing, in: *2019 IEEE 16th International Conference on Mobile Ad Hoc and Sensor Systems (MASS)*, IEEE, 2019, pp. 265–273.
- [16] J. Zhang, M. Ye, Z. Guo, C.-Y. Yen, H.J. Chao, CFR-RL: traffic engineering with reinforcement learning in SDN, *IEEE J. Sel. Areas Commun.* 38 (10) (2020) 2249–2259.
- [17] M. Ye, Y. Hu, J. Zhang, Z. Guo, H.J. Chao, Mitigating routing update overhead for traffic engineering by combining destination-based routing with reinforcement learning, *IEEE J. Sel. Areas Commun.* 40 (9) (2022) 2662–2677.
- [18] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, Q. Miao, Deep reinforcement learning: a survey, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (4) (2022) 5064–5078.
- [19] S.E. Li, Reinforcement learning for sequential decision and optimal control (2023).
- [20] G. Stampa, M. Arias, D. Sánchez-Charles, V. Muntés-Mulero, A. Cabellos, A deep-reinforcement learning approach for software-defined networking routing optimization, *arXiv preprint arXiv:1709.07080*, 2017.
- [21] N.C. Luong, D.T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, D.I. Kim, Applications of deep reinforcement learning in communications and networking: a survey, *IEEE Commun. Surv. Tutorials* 21 (4) (2019) 3133–3174.
- [22] F. Tang, B. Mao, Y. Kawamoto, N. Kato, Survey on machine learning for intelligent end-to-end communication toward 6G: from network access, routing to traffic control and streaming adaptation, *IEEE Commun. Surv. Tutorials* 23 (3) (2021) 1578–1598.
- [23] R. Amin, E. Rojas, A. Aqdas, S. Ramzan, D. Casillas-Perez, J.M. Arco, A survey on machine learning techniques for routing optimization in SDN, *IEEE Access* 9 (2021) 104582–104611.
- [24] Y. Xiao, J. Liu, J. Wu, N. Ansari, Leveraging deep reinforcement learning for traffic engineering: a survey, *IEEE Commun. Surv. Tutorials* 23 (4) (2021) 2064–2097.
- [25] T. Li, K. Zhu, N.C. Luong, D. Niyato, Q. Wu, Y. Zhang, B. Chen, Applications of multi-agent reinforcement learning in future internet: a comprehensive survey, *IEEE Commun. Surv. Tutorials* 24 (2) (2022) 1240–1279.
- [26] R.S. Sutton, A.G. Barto, et al., *Reinforcement Learning: an Introduction*, vol. 1, MIT Press Cambridge, 1998.
- [27] Q. Huang, Model-based or model-free, a review of approaches in reinforcement learning, in: *2020 International Conference on Computing and Data Science (CDS)*, IEEE, 2020, pp. 219–221.
- [28] R.S. Sutton, D. McAllester, S. Singh, Y. Mansour, Policy gradient methods for reinforcement learning with function approximation, *Adv. Neural Inf. Process. Syst.* 12 (1999).
- [29] U.M. Fayyad, G. Piatetsky-Shapiro, P. Smyth, et al., Knowledge discovery and data mining: towards a unifying framework, in: *KDD*, vol. 96, 1996, pp. 82–88.
- [30] C. Shearer, The CRISP-DM model: the new blueprint for data mining, *J. Data Warehous.* 5 (4) (2000) 13–22.

- [31] F. Martínez-Plumed, L. Contreras-Ochando, C. Ferri, J. Hernández-Orallo, M. Kull, N. Lachiche, M.J. Ramírez-Quintana, P. Flach, CRISP-DM twenty years later: from data mining processes to data science trajectories, *IEEE Trans. Knowl. Data Eng.* 33 (8) (2019) 3048–3061.
- [32] G. Salazar-Salazar, M. Mora, H.A. Duran-Limon, F.J. Álvarez Rodríguez, A selective comparative review of CRISP-DM and TDSP development methodologies for big data analytics systems. In *Development Methodologies for Big Data Analytics Systems: Plan-driven, Agile, Hybrid, Lightweight Approaches*, 2023, 161–185.
- [33] A. Medina, N. Taft, K. Salamatian, S. Bhattacharyya, C. Diot, Traffic matrix estimation: existing techniques and new directions, *ACM SIGCOMM Comput. Commun. Rev.* 32 (4) (2002) 161–174.
- [34] M.E. Mustafa, S.A. Talab, The effect of queuing mechanisms first in first out (FIFO), priority queuing (PQ) and weighted fair queuing (WFQ) on network's routers and applications (2016).
- [35] R.J.T. Morris, Priority queuing networks, *Bell Syst. Tech. J.* 60 (8) (1981) 1745–1769.
- [36] A.G. Greenberg, N. Madras, How fair is fair queuing, *J. ACM* 39 (3) (1992) 568–598.
- [37] N. Geng, T. Lan, V. Aggarwal, Y. Yang, M. Xu, A multi-agent reinforcement learning perspective on distributed traffic engineering, in: 2020 IEEE 28th International Conference on Network Protocols (ICNP), IEEE, 2020, pp. 1–11.
- [38] X. Zhao, C. Wu, F. Le, Improving inter-domain routing through multi-agent reinforcement learning, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, 2020, pp. 1129–1134.
- [39] Y. Rekhter, T. Li, S. Hares, A Border Gateway Protocol 4 (BGP-4). Technical Report, 2006.
- [40] A. Mestres, A. Rodriguez-Natal, J. Carner, P. Barlet-Ros, E. Alarcón, M. Solé, V. Muntés-Mulero, D. Meyer, S. Barkai, M.J. Hibbett, et al., Knowledge-defined networking, *ACM SIGCOMM Comput. Commun. Rev.* 47 (3) (2017) 2–10.
- [41] C. Yu, J. Lan, Z. Guo, Y. Hu, DROM: optimizing the routing in software-defined networks with deep reinforcement learning, *IEEE Access* 6 (2018) 64533–64539.
- [42] P. Swain, U. Kamalia, R. Bhandarkar, T. Modi, CODRL: intelligent packet routing in SDN using convolutional deep reinforcement learning, in: 2019 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS), IEEE, 2019, pp. 1–6.
- [43] P. Sun, Y. Hu, J. Lan, L. Tian, M. Chen, TIDE: time-relevant deep reinforcement learning for routing optimization, *Futur. Gener. Comput. Syst.* 99 (2019) 401–409.
- [44] R. McGeer, A safe, efficient update protocol for openflow networks, in: *Proceedings of the First Workshop on Hot Topics in Software Defined Networks*, 2012, pp. 61–66.
- [45] T.A.Q. Pham, Y. Hadjadj-Aoul, A. Outtagarts, Deep reinforcement learning based QoS-aware routing in knowledge-defined networking, in: *Quality, Reliability, Security and Robustness in Heterogeneous Systems: 14th EAI International Conference, Qshine 2018, Ho Chi Minh City, Vietnam, December 3–4, 2018, Proceedings 14*, Springer, 2019, pp. 14–26.
- [46] D.M. Casas-Velasco, O.M.C. Rendon, N.L. da Fonseca, DRSIR: a deep reinforcement learning approach for routing in software-defined networking, *IEEE Trans. Netw. Serv. Manag.* 19 (4) (2021) 4807–4820.
- [47] X. Guo, H. Lin, Z. Li, M. Peng, Deep-reinforcement-learning-based QoS-aware secure routing for SDN-IoT, *IEEE Internet Things J.* 7 (7) (2019) 6242–6251.
- [48] Q. Yan, F.R. Yu, Q. Gong, J. Li, Software-defined networking (SDN) and distributed denial of service (DDoS) attacks in cloud computing environments: a survey, some research issues, and challenges, *IEEE Commun. Surv. Tutorials* 18 (1) (2015) 602–622.
- [49] J. Sen, M.G. Chandra, S.G. Harihar, H. Reddy, P. Balamuralidhar, A mechanism for detection of gray hole attack in mobile AD Hoc networks, in: 2007 6th International Conference on Information, Communications & Signal Processing, IEEE, 2007, pp. 1–5.
- [50] H. Feng, W. Zhang, Y. Liu, Y. Li, J. Yin, J. Kang, GNN-enabled multi-agent DRL for adaptive path selection in multi-network domains, *IEEE Trans. Netw. Sci. Eng.* (2025) 1–16.
- [51] K. Holmberg, D. Yuan, Optimization of internet protocol network design and routing, *Netw.: Int. J.* 43 (1) (2004) 39–53.
- [52] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, et al., P4: programming protocol-independent packet processors, *ACM SIGCOMM Comput. Commun. Rev.* 44 (3) (2014) 87–95.
- [53] C. Liu, M. Xu, Y. Yang, N. Geng, DRL-OR: deep reinforcement learning-based on-line routing for multi-type service requirements, in: IEEE INFOCOM 2021-IEEE Conference on Computer Communications, IEEE, 2021, pp. 1–10.
- [54] T. Dong, Z. Zhuang, Q. Qi, J. Wang, H. Sun, F.R. Yu, T. Sun, C. Zhou, J. Liao, Intelligent joint network slicing and routing via gcn-powered multi-task deep reinforcement learning, *IEEE Trans. Cogn. Commun. Netw.* 8 (2) (2021) 1269–1286.
- [55] X. He, X. Zhuge, F. Dang, W. Xu, Z. Yang, DeepScheduler: enabling flow-aware scheduling in time-sensitive networking, in: IEEE INFOCOM 2023-IEEE Conference on Computer Communications, IEEE, 2023, pp. 1–10.
- [56] A. Nasrallah, A.S. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, H. ElBakoury, Ultra-low latency (ULL) networks: the IEEE TSN and IETF DetNet standards and related 5G ULL research, *IEEE Commun. Surv. Tutorials* 21 (1) (2018) 88–145.
- [57] J. Ren, W. Zhang, H. Wang, D. Yang, S. Wang, H. Zhang, S. Cui, Toward deterministic wide-area networks via deadline-aware routing and scheduling, *IEEE Trans. Netw.* (2025) 1–17.
- [58] N. Kato, B. Mao, F. Tang, Y. Kawamoto, J. Liu, Ten challenges in advancing machine learning technologies toward 6G, *IEEE Wirel. Commun.* 27 (3) (2020) 96–103.
- [59] X. Mai, Q. Fu, Y. Chen, Packet routing with graph attention multi-agent reinforcement learning, in: 2021 IEEE Global Communications Conference (GLOBECOM), IEEE, 2021, pp. 1–6.
- [60] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C.H. Liu, D. Yang, Experience-driven networking: a deep reinforcement learning based approach, in: IEEE INFOCOM 2018-IEEE Conference on Computer Communications, IEEE, 2018, pp. 1871–1879.
- [61] H. Mao, Z. Gong, Y. Ni, Z. Xiao, ACCNet: actor-coordinator-critic net for “learning-to-communicate” with deep multi-agent reinforcement learning, *arXiv preprint arXiv:1706.03235*, 2017.
- [62] H. Mao, Z. Zhang, Z. Xiao, Z. Gong, Modelling the dynamic joint policy of teammates with attention multi-agent DDPG, *arXiv preprint arXiv:1811.07029*, 2018.
- [63] H. Mao, Z. Gong, Z. Zhang, Z. Xiao, Y. Ni, Learning multi-agent communication under limited-bandwidth restriction for internet packet routing, *arXiv preprint arXiv:1903.05561*, 2019.
- [64] H. Mao, Z. Gong, Z. Xiao, Reward design in cooperative multi-agent reinforcement learning for packet routing, *arXiv preprint arXiv:2003.03433*, 2020.
- [65] X. You, X. Li, Y. Xu, H. Feng, J. Zhao, H. Yan, Toward packet routing with fully distributed multiagent deep reinforcement learning, *IEEE Trans. Syst. Man Cybern. Syst.* 52 (2) (2020) 855–868.
- [66] Y. Guo, B. Lin, Q. Tang, Y. Ma, H. Luo, H. Tian, K. Chen, Distributed traffic engineering in hybrid software defined networks: a multi-agent reinforcement learning framework, *IEEE Trans. Netw. Serv. Manag.* (2024).
- [67] X. Li, Y. Xiao, S. Liu, X. Lu, F. Liu, W. Zhou, J. Liu, GAPPO-a graph attention reinforcement learning based robust routing algorithm, in: 2023 IEEE 34th Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC), IEEE, 2023, pp. 1–7.
- [68] M. Ding, Y. Guo, Z. Huang, B. Lin, H. Luo, GROM: a generalized routing optimization method with graph neural network and deep reinforcement learning, *J. Netw. Comput. Appl.* 229 (2024) 103927.
- [69] O. Hope, E. Yoneki, GDDR: GNN-based data-driven routing, in: 2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS), IEEE, 2021, pp. 517–527.
- [70] P. Sun, Z. Guo, J. Li, Y. Xu, J. Lan, Y. Hu, Enabling scalable routing in software-defined networks with deep reinforcement learning on critical nodes, *IEEE/ACM Trans. Netw.* 30 (2) (2021) 629–640.
- [71] T. Dong, Q. Qi, J. Wang, A.X. Liu, H. Sun, Z. Zhuang, J. Liao, Generative adversarial network-based transfer reinforcement learning for routing with prior knowledge, *IEEE Trans. Netw. Serv. Manag.* 18 (2) (2021) 1673–1689.
- [72] W. Wei, H. Gu, L. Fu, B. Li, Grace: toward routing in dynamic network environments with graph embedding, *IEEE Trans. Netw.* (2025) 1–13.
- [73] X. Li, T. Zhang, J. Wang, Z. Han, J. Liu, J. Kang, D. Niyato, A. Jamalipour, Achieving network resilience through graph neural network-enabled deep reinforcement learning, *IEEE Netw.* (2025) 1.
- [74] Y. Song, J. Zhou, X. Li, J. Liu, Enabling robust and generalized DRL-driven routing optimization via exploration of graph neural networks, *IEEE Trans. Cogn. Commun. Netw.* (2025) 1.
- [75] A. Lakhina, K. Papagiannaki, M. Crovella, C. Diot, E.D. Kolaczyk, N. Taft, Structural analysis of network traffic flows, in: *Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems*, 2004, pp. 61–72.
- [76] L. Tassiulas, A. Ephremides, Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks, in: 29th IEEE Conference on Decision and Control, IEEE, 1990, pp. 2130–2132.
- [77] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: *International Conference on Machine Learning*, PMLR, 2017, pp. 1263–1272.
- [78] J. Bruna, W. Zaremba, A. Szlam, Y. LeCun, Spectral networks and locally connected networks on graphs, *arXiv preprint arXiv:1312.6203*, 2013.
- [79] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, *arXiv preprint arXiv:1710.10903*, 2017.
- [80] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, Y. Sun, Masked label prediction: unified message passing model for semi-supervised classification, *arXiv preprint arXiv:2009.03509*, 2020.
- [81] B. Perozzi, R. Al-Rfou, S. Skiena, DeepWalk: online learning of social representations, in: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '14, New York, NY, USA. Association for Computing Machinery, 2014, pp. 701–710.
- [82] S. Munikoti, D. Agarwal, L. Das, M. Halappanavar, B. Natarajan, Challenges and opportunities in deep reinforcement learning with graph neural networks: a comprehensive review of algorithms and applications, *IEEE Trans. Neural Netw. Learn. Syst.* (2023).
- [83] K.L. Wagstaff, G. Doran, A. Davies, S. Anwar, S. Chakraborty, M. Cameron, I. Daubar, C. Phillips, Enabling onboard detection of events of scientific interest for the Europa clipper spacecraft, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2191–2201.
- [84] U. Challita, H. Ryden, H. Tullberg, When machine learning meets wireless cellular networks: deployment, challenges, and applications, *IEEE Commun. Mag.* 58 (6) (2020) 12–18.
- [85] A. Carrio, C. Sampedro, A. Rodriguez-Ramos, P. Campoy, A review of deep learning methods and applications for unmanned aerial vehicles, *J. Sens.* 2017 (1) (2017) 3296874.
- [86] Y. Bao, Y. Peng, C. Wu, Deep learning-based job placement in distributed machine learning clusters, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 505–513.

- [87] P. Kumar, Y. Yuan, C. Yu, N. Foster, R. Kleinberg, P. Lapukhov, C.L. Lim, R. Soulé, *Semi-oblivious traffic engineering: the road not taken*, in: 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), 2018, pp. 157–170.
- [88] H. Mao, M. Schwarzkopf, H. He, M. Alizadeh, Towards safe online reinforcement learning in computer systems, in: NeurIPS Machine Learning for Systems Workshop, Curran Associates New York, NY, 2019.
- [89] C. Liu, V. Aggarwal, T. Lan, N. Geng, Y. Yang, M. Xu, Q. Li, FERN: leveraging graph attention networks for failure evaluation and robust network design, *IEEE/ACM Trans. Netw.* 32 (2) (2023) 1003–1018.
- [90] J. Suárez-Varela, P. Almasan, M. Ferriol-Galmés, K. Rusek, F. Geyer, X. Cheng, X. Shi, S. Xiao, F. Scarselli, A. Cabellos-Aparicio, et al., Graph neural networks for communication networks: context, use cases and opportunities, *IEEE Netw.* 37 (3) (2022) 146–153.
- [91] Y. Guo, W. Wang, H. Zhang, W. Guo, Z. Wang, Y. Tian, X. Yin, J. Wu, Traffic engineering in hybrid software defined network via reinforcement learning, *J. Netw. Comput. Appl.* 189 (2021) 103116.
- [92] R. Nair, M. Tambe, M. Yokoo, D. Pynadath, S. Marsella, Taming decentralized POMDPs: towards efficient policy computation for multiagent settings, in: *IJCAI*, vol. 3, 2003, pp. 705–711.
- [93] W.Y.B. Lim, N.C. Luong, D.T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, C. Miao, Federated learning in mobile edge networks: a comprehensive survey, *IEEE Commun. Surv. Tutorials* 22 (3) (2020) 2031–2063.
- [94] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fiedland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [95] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, *arXiv preprint arXiv:1509.02971*, 2015.
- [96] D. Pathak, P. Agrawal, A.A. Efros, T. Darrell, Curiosity-driven exploration by self-supervised prediction, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2017, pp. 488–489.
- [97] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint arXiv:1707.06347*, 2017.
- [98] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *International Conference on Machine Learning*, Pmlr, 2018, pp. 1861–1870.
- [99] A. Varga, Discrete event simulation system, in: *Proc. Of the European Simulation Multiconference (ESM'2001)*, vol. 17, 2001.
- [100] P. Gawłowicz, A. Zubow, NS-3 meets openAI gym: the playground for machine learning in networking research, in: *Proceedings of the 22nd International ACM Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, 2019, pp. 113–120.
- [101] B. Lantz, B. Heller, N. McKeown, A network in a laptop: rapid prototyping for software-defined networks, in: *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, 2010, pp. 1–6.
- [102] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, OpenFlow: enabling innovation in campus networks, *ACM SIGCOMM Comput. Commun. Rev.* 38 (2) (2008) 69–74.
- [103] J. Medved, R. Varga, A. Tkacik, K. Gray, OpenDayLight: towards a model-driven SDN controller architecture, in: *Proceeding of IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks 2014*, IEEE, 2014, pp. 1–6.
- [104] P. Floodlight, Floodlight. *Disponível em*, 2021, <http://www.projectfloodlight.org/floodlight>.
- [105] RYU Project Team and others, Ryu SDN Framework, RYU Project Team, 2014.
- [106] S. Kaur, J. Singh, N.S. Ghuman, Network programmability using pox controller, in: *ICCCS International Conference on Communication, Computing & Systems*, IEEE, vol. 138, Sn, 2014, pp. 70.
- [107] R. Albert, A.-L. Barabási, Statistical mechanics of complex networks, *Rev. Mod. Phys.* 74 (1) (2002) 47.
- [108] A. Medina, A. Lakhina, I. Matta, J. Byers, Brite: an approach to universal topology generation, in: *MASCOTS 2001, Proceedings Ninth International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*, IEEE, 2001, pp. 346–353.
- [109] N. Spring, R. Mahajan, D. Wetherall, Measuring ISP topologies with rocketfuel, *ACM SIGCOMM Comput. Commun. Rev.* 32 (4) (2002) 133–145.
- [110] S. Knight, H.X. Nguyen, N. Falkner, R. Bowden, M. Roughan, The internet topology zoo, *IEEE J. Sel. Areas Commun.* 29 (9) (2011) 1765–1775.
- [111] A.H. Lashkari, G.D. Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: *International Conference on Information Systems Security and Privacy*, vol. 2, SciTePress, 2017, pp. 253–262.
- [112] J. Rehn, T. Barras, D. Bonacorsi, J. Hernandez, I. Semeniov, L. Tuura, Y. Wu, Phedex high-throughput data transfer management system, in: *Computing in High Energy and Nuclear Physics (CHEP)*, vol. 2006, 2006.
- [113] TMgen traffic matrix generator, <https://github.com/progwriter/TMgen>. (Accessed 2018).
- [114] N. Bonelli, S. Giordano, G. Prociassi, R. Secchi, et al., Brute: a high performance and extensible traffic generator. In *Proc. of SPECTS 2005*, volume 1, 2005, 222–227.
- [115] iperf, <https://iperf.fr/>. (Accessed March 2022).
- [116] Ostinato network traffic generator and analyzer, <http://ostinato.org/>. (Accessed July 2017).
- [117] F. Fidler, J. Wilcox, Reproducibility of scientific results (2018).
- [118] R. Peng, The reproducibility crisis in science: a statistical counterattack, *Significance* 12 (3) (2015) 30–32.
- [119] R. Lowe, Y.I. Wu, A. Tamar, O. Pieter Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [120] X. Chu, H. Ye, Parameter sharing deep deterministic policy gradient for co-operative multi-agent reinforcement learning, *arXiv preprint arXiv:1710.00336*, 2017.
- [121] J. Zhou, M. Tewari, M. Zhu, A. Kabbani, L. Poutievski, A. Singh, A. Vahdat, WCMF: weighted cost multipathing for improved fairness in data centers, in: *Proceedings of the Ninth European Conference on Computer Systems*, 2014, pp. 1–14.
- [122] Y. Wang, X. Tao, Q. He, Y. Kuang, A dynamic load balancing method of cloud-center based on SDN, *China Commun.* (2016).
- [123] S.-C. Lin, I.F. Akyildiz, P. Wang, M. Luo, QoS-aware adaptive routing in multi-layer hierarchical software defined networks: a reinforcement learning approach, in: 2016 IEEE International Conference on Services Computing (SCC), IEEE, 2016, pp. 25–33.
- [124] S. Zeng, X. Xu, Y. Chen, Multi-agent reinforcement learning for adaptive routing: a hybrid method using eligibility traces, in: 2020 IEEE 16th International Conference on Control & Automation (ICCA), IEEE, 2020, pp. 1332–1339.
- [125] D. Mukhutinov, A. Filchenkov, A. Shalyto, V. Vyatkin, Multi-agent deep learning for simultaneous optimization for time and energy in distributed routing system, *Futur. Gener. Comput. Syst.* 94 (2019) 587–600.
- [126] A deep-reinforcement learning approach for software-defined networking routing optimization, <http://knowledgedefinednetworking.org>. [Online].
- [127] DRSIR: a deep reinforcement learning approach for routing in software-defined networking, <https://github.com/danielaCasasv/DRSIR>. [Online].
- [128] Google cloud topology, <https://cloud.google.com/about/locations/>. [Online].
- [129] U. Feige, P. Raghavan, Exact analysis of hot-potato routing, in: *Proceedings, 33rd Annual Symposium on Foundations of Computer Science*, IEEE Computer Society, 1992, pp. 553–562.
- [130] DRL-OR: deep reinforcement learning-based online routing for multi-type service requirements, <https://github.com/netlab-lcy/DRL-OR>. [Online].
- [131] S. Vissicchio, L. Vanbever, O. Bonaventure, Opportunities and research challenges of hybrid software defined networks, *ACM SIGCOMM Comput. Commun. Rev.* 44 (2) (2014) 70–75.
- [132] S. Agarwal, M. Kodialam, T.V. Lakshman, Traffic engineering in software defined networks, in: 2013 Proceedings IEEE INFOCOM, IEEE, 2013, pp. 2211–2219.
- [133] X.F. Wang, G. Chen, Pinning control of scale-free dynamical networks, *Physica A: Stat. Mech. Appl.* 310 (3–4) (2002) 521–531.
- [134] Y.-Y. Liu, J.-J. Slotine, A.-L. Barabási, Controllability of complex networks, *Nature* 473 (7346) (2011) 167–173.
- [135] L. Wang, X. Zhang, H. Su, J. Zhu, A comprehensive survey of continual learning: theory, method and application, *IEEE Trans. Pattern Anal. Mach. Intell.* (2024).
- [136] Y. Xiao, J. Li, J. Wu, J. Liu, On design and implementation of reinforcement learning based cognitive routing for autonomous networks, *IEEE Commun. Lett.* 27 (1) (2022) 205–209.
- [137] R.W. Thomas, D.H. Friend, L.A. DaSilva, A.B. MacKenzie, Cognitive networks, *Cogn. Radio, Softw. Defin. Radio, Adapt. Wirel. Syst.* (2007) 17–41.
- [138] E. Tzeng, J. Hoffman, K. Saenko, T. Darrell, Adversarial discriminative domain adaptation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 7167–7176.
- [139] M. Karakas, A. Duresi, A survey: control plane scalability issues and approaches in software-defined networking (SDN), *Comput. Netw.* 112 (2017) 279–293.
- [140] A. Abuarqoub, A review of the control plane scalability approaches in software defined networking, *Future Internet* 12 (3) (2020) 49.
- [141] T. Wang, X. Bao, I. Clavera, J. Hoang, Y. Wen, E. Langlois, S. Zhang, G. Zhang, P. Abbeel, J. Ba, Benchmarking model-based reinforcement learning, *arXiv preprint arXiv:1907.02057*, 2019.
- [142] A. HasanadeZonuz, D. Kalathil, S. Shakkottai, Reinforcement learning for multi-hop scheduling and routing of real-time flows, in: 2020 18th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOPT), IEEE, 2020, pp. 1–8.
- [143] A. Panda, C. Scott, A. Ghodsi, T. Koponen, S. Shenker, Cap for networks, in: *Proceedings of the Second ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking*, 2013, pp. 91–96.
- [144] Z. Zhang, L. Ma, K. Poularakis, K.K. Leung, L. Wu, DQ scheduler: deep reinforcement learning based controller synchronization in distributed SDN, in: *ICC 2019-2019 IEEE International Conference on Communications (ICC)*, IEEE, 2019, pp. 1–7.
- [145] P. Berde, M. Gerola, J. Hart, Y. Higuchi, M. Kobayashi, T. Koide, B. Lantz, B. O'Connor, P. Radoslavov, W. Snow, et al., ONOS: towards an open, distributed SDN OS, in: *Proceedings of the Third Workshop on Hot Topics in Software Defined Networking*, 2014, pp. 1–6.
- [146] M. Koren, S. Alsaif, R. Lee, M.J. Kochenderfer, Adaptive stress testing for autonomous vehicles, in: 2018 IEEE Intelligent Vehicles Symposium (IV), IEEE, 2018, pp. 1–7.
- [147] C.-H.-H. Yang, J. Qi, P.-Y. Chen, Y. Ouyang, I.-T.D. Hung, C.-H. Lee, X. Ma, Enhanced adversarial strategically-timed attacks against deep reinforcement learning, in: *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2020, pp. 3407–3411.
- [148] L. Baier, F. Jöhren, S. Seebacher, Challenges in the deployment and operation of machine learning in practice, in: *ECIS*, vol. 1, 2019.
- [149] U. Bhatt, A. Xiang, S. Sharma, A. Weller, A. Taly, Y. Jia, J. Ghosh, R. Puri, J.M. Moura, P. Eckersley, Explainable machine learning in deployment, in: *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 2020, pp. 648–657.
- [150] D. Lyu, F. Yang, B. Liu, S. Gustafson, SDRL: interpretable and data-efficient deep reinforcement learning leveraging symbolic planning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 2970–2977.

- [151] A. Verma, V. Murali, R. Singh, P. Kohli, S. Chaudhuri, Programmatically interpretable reinforcement learning, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 5045–5054.
- [152] C. Glanois, P. Weng, M. Zimmer, D. Li, T. Yang, J. Hao, W. Liu, A survey on interpretable reinforcement learning, *Mach. Learn.* 113 (8) (2024) 5847–5890.
- [153] Z. Zhu, K. Lin, A.K. Jain, J. Zhou, Transfer learning in deep reinforcement learning: a survey, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (11) (2023) 13344–13362.
- [154] W.Y.B. Lim, N.C. Luong, D.T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, C. Miao, Federated learning in mobile edge networks: a comprehensive survey, *IEEE Commun. Surv. Tutorials* 22 (3) (2020) 2031–2063.
- [155] H. Li, Y. Liu, X. Zhou, X. Vasilakos, R. Nejabati, S. Yan, D. Simeonidou, Adaptive resource management for edge network slicing using incremental multi-agent deep reinforcement learning, *arXiv preprint arXiv:2310.17523*, 2023.
- [156] Y. Zheng, L. Lin, T. Zhang, H. Chen, Q. Duan, Y. Xu, X. Wang, Enabling robust DRL-driven networking systems via teacher-student learning, *IEEE J. Sel. Areas Commun.* 40 (1) (2021) 376–392.
- [157] H. Du, R. Zhang, Y. Liu, J. Wang, Y. Lin, Z. Li, D. Niyato, J. Kang, Z. Xiong, S. Cui, B. Ai, H. Zhou, D.I. Kim, Enhancing deep reinforcement learning: a tutorial on generative diffusion models in network optimization, *IEEE Commun. Surv. Tutorials* 26 (4) (2024) 2611–2646.
- [158] Z. Liu, H. Du, J. Lin, Z. Gao, L. Huang, S. Hosseinalipour, D. Niyato, DNN partitioning, task offloading, and resource allocation in dynamic vehicular networks: a lyapunov-guided diffusion-based reinforcement learning approach, *IEEE Trans. Mob. Comput.* 24 (3) (2025) 1945–1962.

Author biography

Yuqian Song received the B.E. degree (Hons.) in communication engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 2021, where she is currently pursuing the Ph.D. degree with the School of Artificial Intelligence. Her current research interests include network traffic engineering, wireless resource allocation, vehicular networks, and reinforcement learning.

Mao Ni received his Master's degree in Computer Technology from Beijing University of Posts and Telecommunications (BUPT) in 2012. Currently, he serves as a Product Development Lead at China Mobile Communications Group Terminal Co., Ltd., where he has spearheaded multiple large-scale R&D initiatives for China Mobile and contributed to national-level key projects in Computing Force Network. With over 10 patents to his name, his research focuses on Computing Force Network, 5G terminal technology, operating systems, and artificial intelligence.

Jun Liu (Member, IEEE) received the B.E. and Ph.D. degrees from the Department of Information Engineering, Beijing University of Posts and Telecommunications (BUPT), in 1998 and 2003, respectively. He is currently the Director of the Research Center for Data Science, School of Artificial Intelligence, BUPT. His research interests include telecommunication big data analysis and network management.